

Tema 8.2: Pruebas en Android



Pruebas en Android

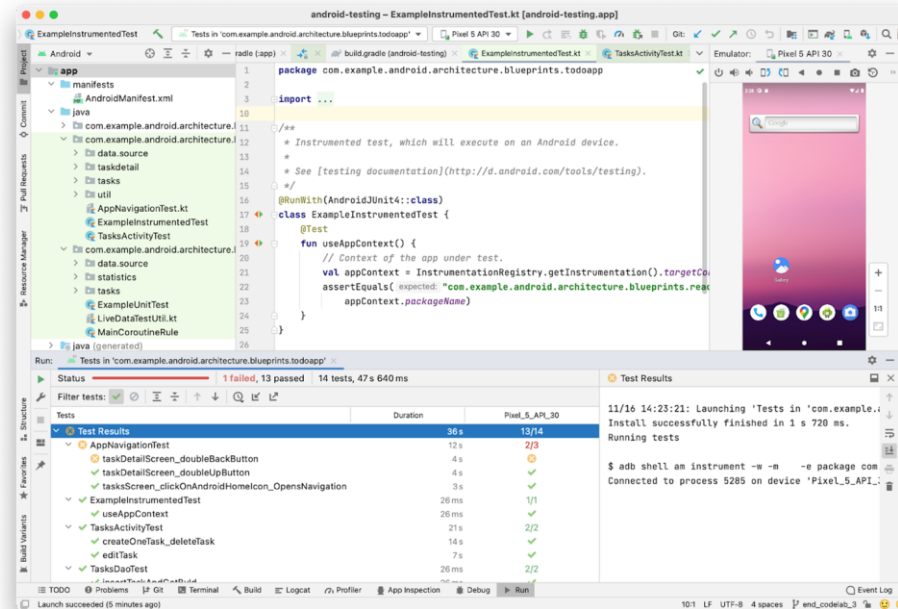
Las **pruebas** son una **parte integral** del proceso de desarrollo de aplicaciones. Al ejecutar pruebas en las aplicaciones de manera constante, se puede verificar la corrección, el comportamiento funcional y la usabilidad de las aplicaciones antes de lanzarlas públicamente.

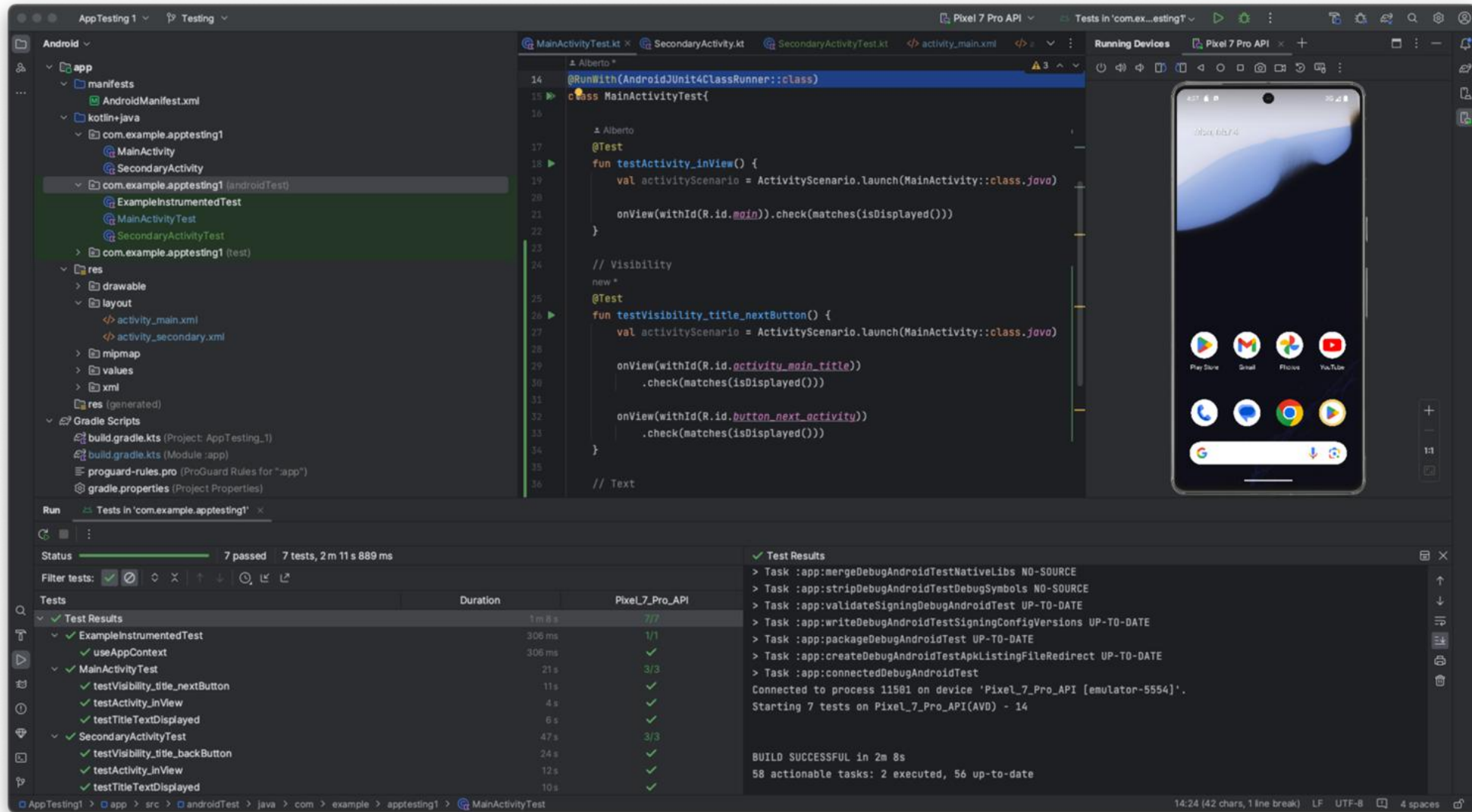
Podemos **probar manualmente** las aplicaciones navegando a través de ellas. Se pueden utilizar diferentes dispositivos y emuladores, cambiar el idioma del sistema e intentar generar cada error de usuario o recorrer cada flujo de usuario.

Sin embargo, las pruebas manuales no escalan bien y puede ser fácil pasar por alto las regresiones en el comportamiento de las aplicaciones. Las **pruebas automatizadas** implican el uso de herramientas que realizan pruebas por nosotros, lo cual es más rápido, más repetible y, en general, nos brinda comentarios más prácticos sobre las aplicaciones en una etapa más temprana del proceso de desarrollo.

Pruebas en Android

Android Studio está **diseñado** para **simplificar las pruebas**. Contiene muchas funciones para simplificar la forma de crear, ejecutar y analizar pruebas. Podemos configurar pruebas que se ejecuten en su máquina local o pruebas instrumentadas que se ejecuten en un dispositivo. Podemos ejecutar fácilmente **una única prueba** o un **grupo específico de pruebas** en uno o más dispositivos. Los resultados de la prueba se muestran directamente dentro de Android Studio.





Android Studio muestra una descripción general de los resultados de las pruebas.

Pruebas en Android

Las pruebas también ofrecen las siguientes **ventajas**:

- ✓ **Avisos** rápidos sobre fallos.
- ✓ **Detección** temprana de fallos en el ciclo de desarrollo.
- ✓ **Refactorización de código** más seguro, que te permite optimizar el código sin preocuparte por las regresiones.
- ✓ **Velocidad de desarrollo** estable, que ayuda a minimizar la deuda técnica

Pruebas en Android

Las aplicaciones para dispositivos móviles son complejas y deben funcionar bien en muchos entornos. Por lo tanto, hay muchos tipos de pruebas.

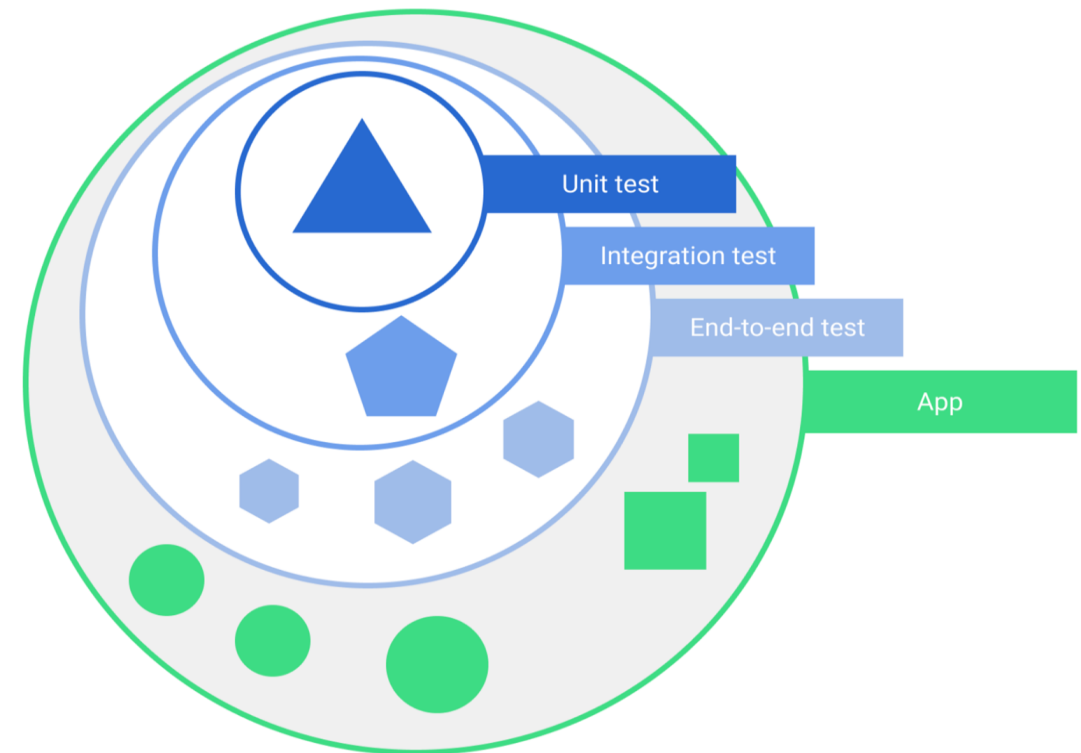
Por ejemplo, hay diferentes **tipos de pruebas** según el tema:

- **Pruebas funcionales:** ¿Mi app hace lo que se supone que debe hacer?
- **Pruebas de rendimiento:** ¿lo hace de forma rápida y eficiente?
- **Pruebas de accesibilidad:** ¿Funciona bien con los servicios de accesibilidad?
- **Pruebas de compatibilidad:** ¿Funciona bien en todos los dispositivos y niveles de API?

Pruebas en Android Studio

Las pruebas también varían según el tamaño o el grado de aislamiento:

- Las **pruebas de unidades o pequeñas** solo verifican una parte muy pequeña de la app, como un método o una clase.
- Las **pruebas de nivel intermedio** se encuentran en el medio y verifican la integración entre dos o más unidades.
- Las **pruebas de extremo a extremo** o las pruebas importantes verifican partes más grandes de la app al mismo tiempo, como una pantalla completa o un flujo de usuarios.



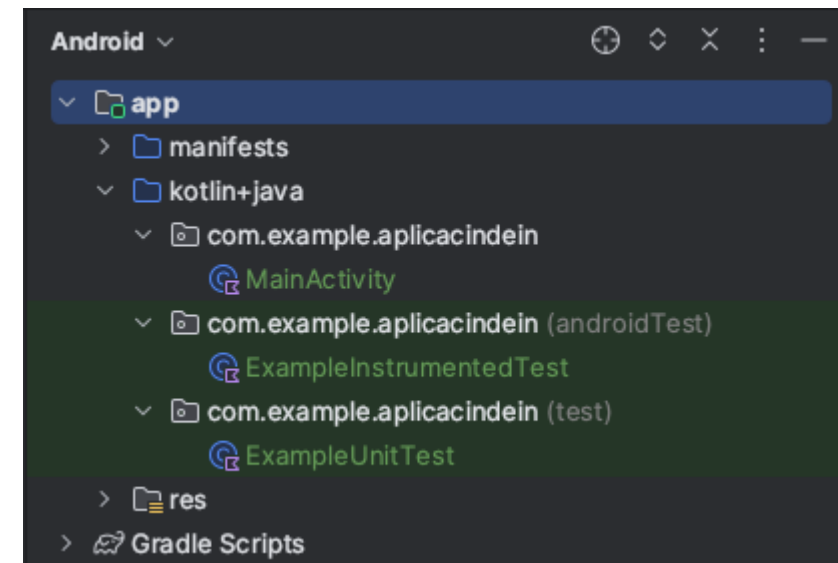
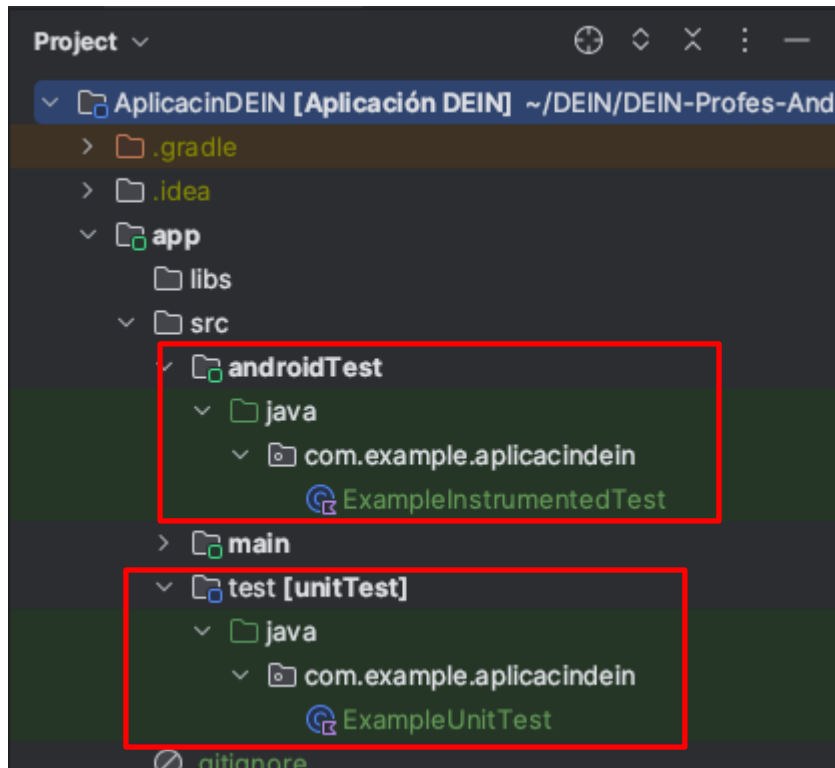
¿Dónde se crean las Pruebas?

Organización de directorios de prueba

- Un proyecto típico de Android Studio contiene **dos directorios** que contienen **pruebas** según su entorno de ejecución. Organiza las pruebas en los siguientes directorios como se describe:
- El directorio **androidTest** debe contener las pruebas que se ejecutan en **dispositivos reales o virtuales**. Estas pruebas incluyen **pruebas de integración, de extremo a extremo** y otras en las que la JVM (Máquina Virtual Java) por sí sola no puede validar la funcionalidad de tu app.
- El directorio **test** debe contener las pruebas que se ejecutan en tu **máquina local**, como las **pruebas unitarias o de unidades**. A diferencia de lo anterior, estas pueden ser pruebas que se ejecutan en una JVM local.

¿Dónde se crean las Pruebas?

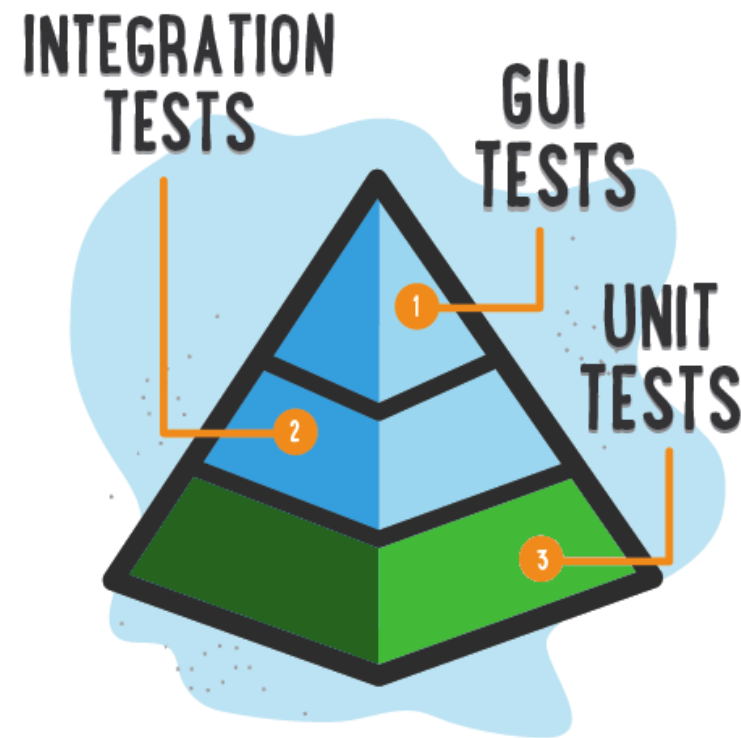
Organización de directorios de prueba en “Project” y “Android”:



Tipos de Pruebas en Android

1. Pruebas de unidades esenciales
(Test Unitarios)

2. Pruebas de interfaz de usuario/UI,
Test de instrumentación o Pruebas
Instrumentales



Pruebas de unidades esenciales (Test Unitarios)

Cómo compilar pruebas Unitarias

Una prueba local **se ejecuta directamente en el programa** en lugar de un dispositivo o emulador de Android. Las pruebas locales te permiten **evaluar la lógica de tu app más rápido**. Sin embargo, no poder interactuar con el framework de Android crea una limitación en los tipos de pruebas que puedes ejecutar.

Una **prueba de unit** verifica el comportamiento de una **pequeña sección de código**, la unidad que se está probando. Para ello, ejecuta el código y verifica el resultado. Suelen ser simples, pero su configuración puede causar problemas cuando la unidad que se está probando no se diseñó teniendo en cuenta la capacidad de prueba, por ejemplo, **el código que deseas verificar debe ser accesible desde una prueba** (no puedes probar un método privado de forma directa, en su lugar, debes probar la clase con sus API públicas).

Cómo compilar pruebas Unitarias

Los archivos de origen para las pruebas de unidades locales se colocan en:

module-name/src/test/ Este directorio ya existe cuando creas un proyecto nuevo con Android Studio.

También debes **configurar las dependencias de prueba** para que tu proyecto use las APIs estándar que proporciona el framework de pruebas **JUnit** u otras como [Truth](#).

Para ello, abre el archivo **build.gradle** del módulo de tu **app** y especifica las siguientes bibliotecas como dependencias. Usa la función `testImplementation` para indicar que se aplican al conjunto de orígenes de prueba local y no a la aplicación:

Preparándonos para pruebas Unitarias

Según la [documentación de Android](#) tendremos que añadir varias dependencias al Gradle. [Versiones](#)

```
dependencies {  
  
    implementation("androidx.core:core-ktx:1.12.0")  
    implementation("androidx.appcompat:appcompat:1.6.1")  
    implementation("com.google.android.material:material:1.11.0")  
    implementation("androidx.constraintlayout:constraintlayout:2.1.4")  
  
    testImplementation("junit:junit:4.13.2")  
    testImplementation("com.google.truth:truth:1.4.2")  
  
    androidTestImplementation("androidx.test.espresso:espresso-core:3.5.1")  
  
    androidTestImplementation("androidx.test:runner:1.5.2")  
    androidTestImplementation("androidx.test:core:1.5.0")  
  
    androidTestImplementation("androidx.test.ext:junit:1.1.5")  
}
```

Cómo crear una clase de prueba de unidades local

Escribe la clase de prueba de unidades local como una **clase de prueba JUnit 4**.

Para ello, **crea una clase** que contenga uno o más **métodos de prueba**, por lo general, en **module-name/src/test/**.

Un método de prueba comienza con la anotación **@Test** y contiene el código para ejecutar y verificar un solo aspecto del componente que quieres probar.

En el siguiente ejemplo, vamos a crear **una clase que valida si el login de un usuario es correcto**. Para ello, comprobaremos que el username no está vacío y que no existe, que password y confirmedPassword coinciden, y que la password contiene más de dos caracteres.

Cómo crear una clase de prueba de unidades local

Para realizar las pruebas vamos a crear primero una **clase de prueba** en el mismo directorio donde se encuentra MainActivity.kt, llámale por ejemplo “UtilidaRegistro”, y pega el siguiente código:

```
package com.example.apptesting2

object UtilidadRegistro {

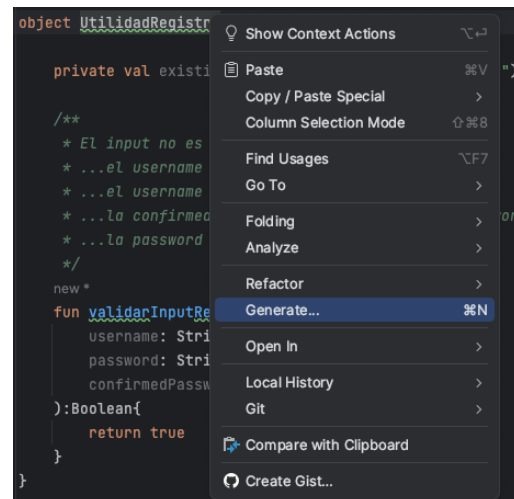
    private val usuariosExistentes = listOf("Alberto","David")

    /**
     * El input no es válido si...
     * ...el username está vacío
     * ...el username existe previamente
     * ...la confirmedPassword no es igual que la password
     * ...la password es menor de 2 dígitos
     */
    fun validarInputRegistro(
        username: String,
        password: String,
        confirmedPassword: String
    ): Boolean{
        return true
    }
}
```


Crear nuevas pruebas Unitarias

Ahora para agregar una **nueva prueba** para una clase o método específico directamente desde tu código fuente sigue estos pasos:

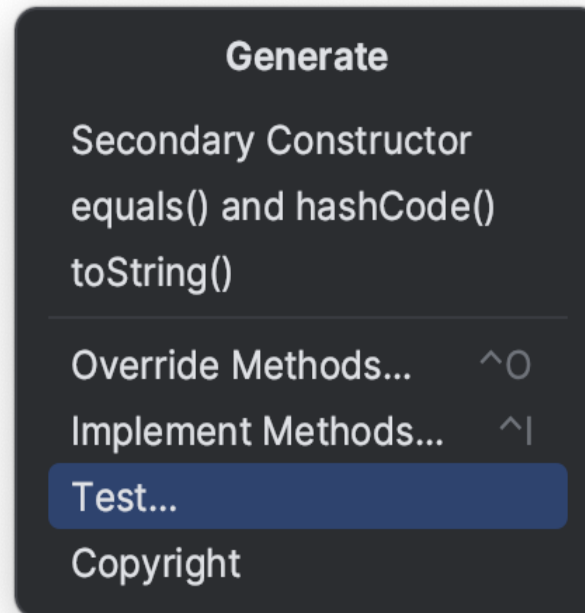
1. **Abre el archivo** fuente que contiene el código que desea probar. (ej. UtilidadRegistro)
2. Coloca el cursor en el nombre de la **clase o método que desea probar** y presiona **Control+Shift+T** (⌘+N macOS).



Crear nuevas pruebas Unitarias

Para agregar una nueva prueba para una clase o método específico directamente desde tu código fuente sigue estos pasos:

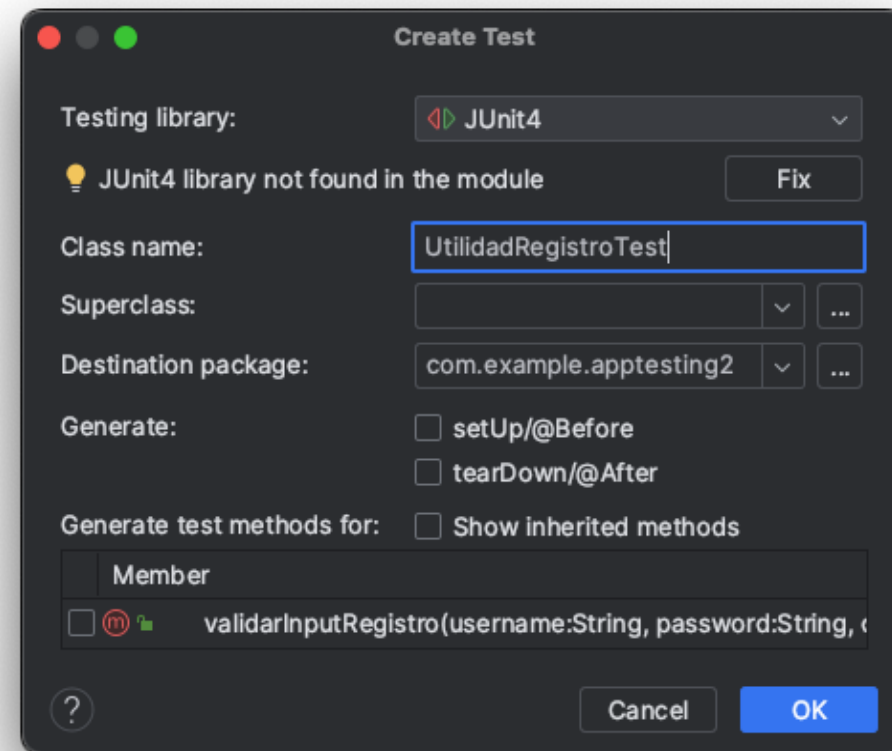
3. En la ventana emergente que aparece, haz clic en **Test...**



Crear nuevas pruebas Unitarias

Para agregar una nueva prueba para una clase o método específico directamente desde tu código fuente sigue estos pasos:

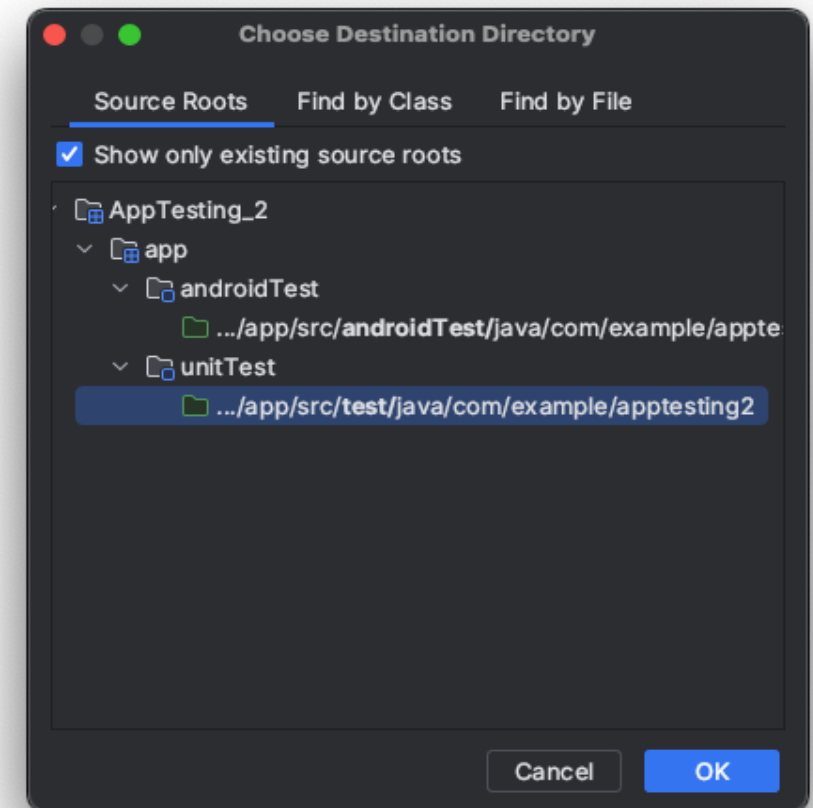
4. En el cuadro de diálogo Crear prueba , elije **JUnit4** y luego haga clic en Aceptar .



Crear nuevas pruebas Unitarias

Para agregar una nueva prueba para una clase o método específico directamente desde tu código fuente sigue estos pasos:

5. En el cuadro de diálogo “**Elegir directorio de destino**”, haz clic en el conjunto de fuentes correspondiente al tipo de prueba que desea crear: “**unitTest**” para una prueba unitaria o “**androidTest**” para una prueba instrumentada. Luego haz clic en **Aceptar** .



Crear nuevas pruebas Unitarias

Alternativamente, puedes crear un archivo de prueba genérico en el conjunto de fuentes de prueba apropiado de la siguiente manera:

1. En la ventana del **Proyecto** a la izquierda, haz clic en el menú desplegable y seleccione la **vista de Android**.
2. Haz clic derecho en el directorio **java** y seleccione **Nuevo > Clase/Archivo Kotlin**.
Alternativamente, puedes seleccionar el directorio java y usar el acceso directo Control+N(Command+N en macOS).
3. En el cuadro de diálogo “Elegir directorio de destino”, haz clic en el conjunto de fuentes correspondiente al tipo de prueba que desea crear: **“unitTest”** para una prueba unitaria o **“androidTest”** para una prueba instrumentada. Luego haz clic en Aceptar.
4. Asigna un nombre al archivo y luego haz clic en Aceptar.

Primera prueba unitaria

Creada la clase para realizar las pruebas, ya podemos programar nuestra primera prueba.

- Hemos cambiado la librería que se importa por defecto (import org.junit.Assert.*) para usar el método **assertThat** de la **librería Truth** de Google.
- Los nombres de las funciones tienen que ser **descriptivos**, incluso se utilizan las (``).
- El caso de uso llama al método que queremos probar, con el username vacío.

```
package com.example.apptesting2

import com.google.common.truth.Truth.assertThat
import org.junit.Test

class UtilidadRegistroTest{

    @Test
    fun `usuario vacio devuelve falso`() {
        val result = UtilidadRegistro.validarInputRegistro(
            "",
            "123",
            "123"
        )
        assertThat(result).isFalse()
    }
}
```

Ejecutar pruebas

Primero, asegúrate de que tu proyecto esté completamente sincronizado con **Gradle**. 

- Para **ejecutar todas las pruebas en un directorio o archivo**, abra la ventana **Proyecto** y realice una de las siguientes acciones:
 - ☐ Haz clic derecho en un directorio o archivo y haga clic en **Run/Ejecutar**. 
 - ☐ Selecciona el directorio o archivo y use el acceso directo Ctrl+Shift+R.

Para **ejecutar todas las pruebas en una clase o un método específico**, abra el archivo de prueba en el Editor de código y realice una de las siguientes acciones:

- ☐ Presiona el ícono **Ejecutar prueba** en el **gutter/canal**. 
- ☐ Haz clic derecho en la clase o método de prueba y haga clic en **Run/Ejecutar** 
- ☐ Selecciona la clase o método de prueba y use el acceso directo Ctrl+Shift+R.



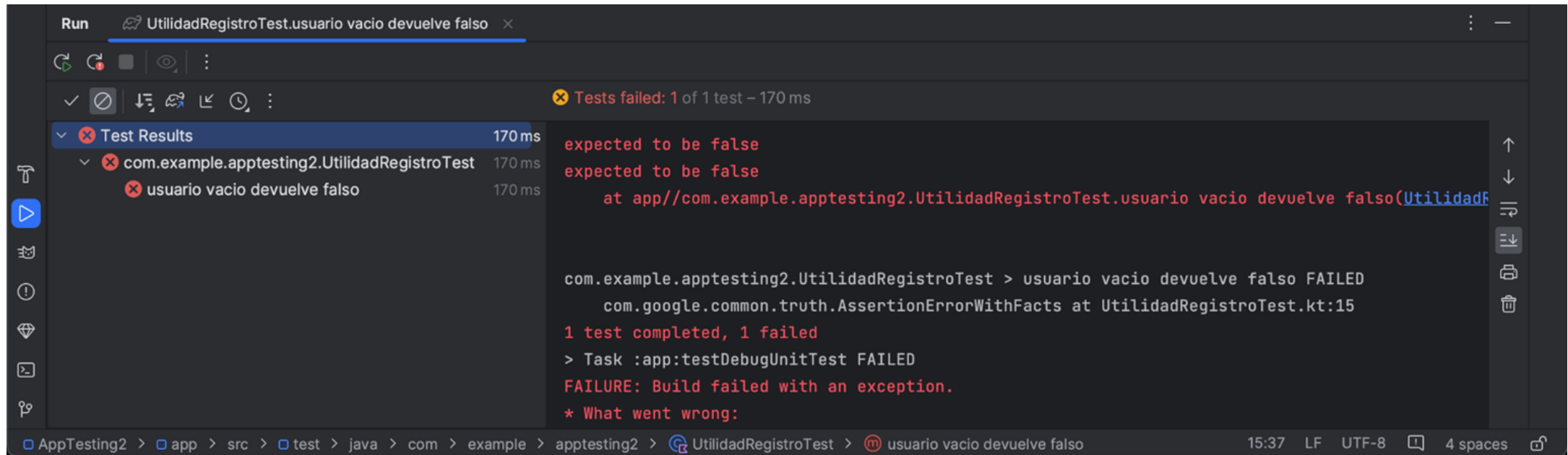
```
1 package com.  
2  
3 > import ...  
14  
15 @RunWith(And  
16 > class MainAc  
17  
18 //Carga  
19  
20 > @Test  
21 fun test  
22 val  
23  
24 onVi  
25 }
```

Sus pruebas instrumentadas se ejecutarán en un dispositivo físico o emulador.

Resultado de las pruebas

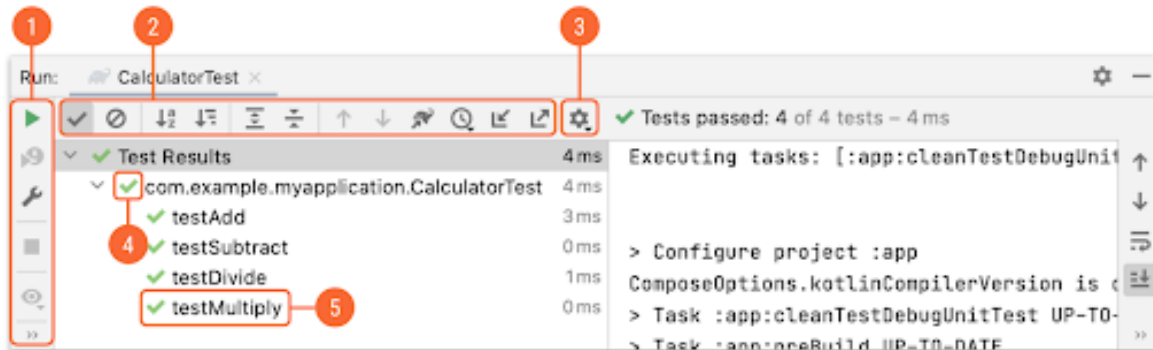
Una vez realizadas las pruebas, siempre se realizarán en orden alfabético (puede ser distinto al orden de implementación en la clase de Test), podemos ver los resultados en la ventana de ejecución.

¿Por qué falla?



Ver resultados de las pruebas

Cuando se ejecutan una o más pruebas desde Android Studio, los resultados aparecen en la ventana Ejecutar:



1. Barra de herramientas de ejecución para volver a ejecutar la prueba actual, detener la prueba actual, volver a ejecutar las pruebas fallidas

1.Barra de herramientas de prueba para filtrar y ordenar los resultados de la prueba.

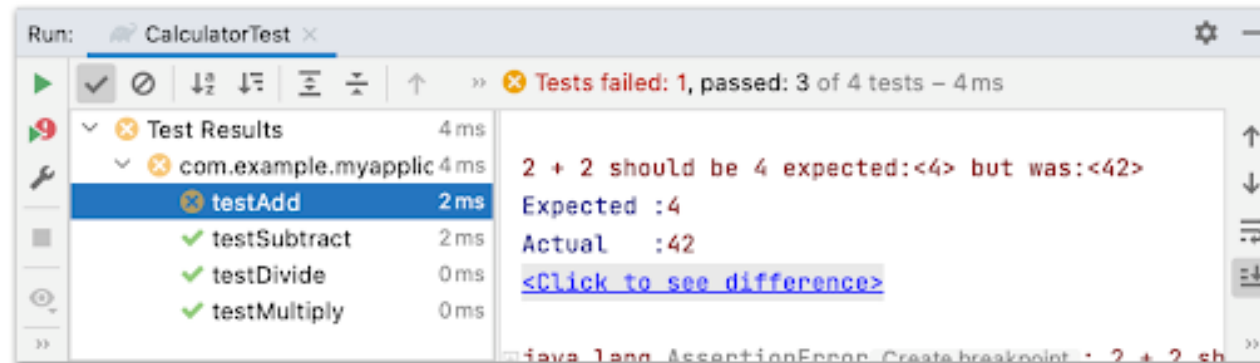
1.Menú contextual para realizar un seguimiento de la prueba en ejecución, mostrar estadísticas en línea, desplazarse hasta el seguimiento de la pila..

1.Los iconos de estado de la prueba indican si una prueba tiene un error, se ignoró, falló, está en progreso, está en pausa, finalizó o no se ejecutó.

1.Haz clic con el botón derecho en una línea en la vista de árbol para mostrar un menú contextual que le permite ejecutar las pruebas en modo de depuración, abrir el archivo de código fuente de prueba o saltar a la línea del código fuente que se está probando.

Ver resultados de las pruebas

Cuando una o más pruebas fallan, la ventana de resultados muestra una señal de advertencia y la cantidad de fallos:

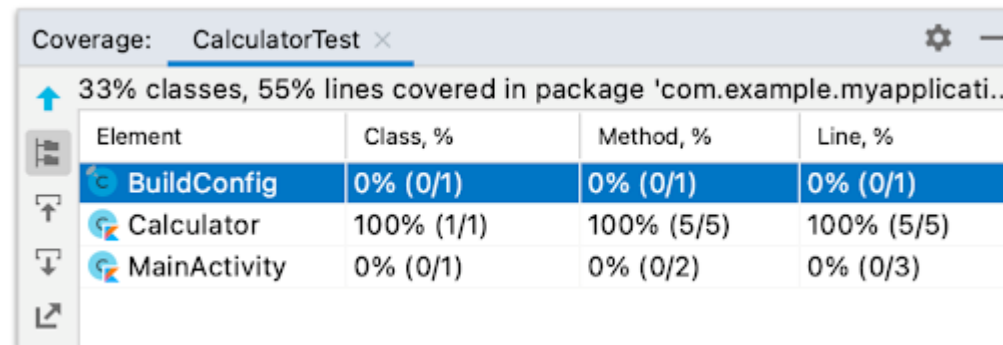


Cuando haces clic en la prueba fallida en la vista de árbol de la izquierda, el panel de salida de la derecha muestra los detalles de esa prueba. Muestra el valor esperado junto al valor real, para que puedas compararlos. El enlace [<Click to see difference>](#) abre un visor de diferencias donde puede ver los resultados uno al lado del otro.

Ver cobertura de las pruebas

La herramienta de **cobertura de pruebas** está disponible para que las pruebas unitarias locales realicen un seguimiento del **porcentaje y las áreas del código** de una aplicación que hayan cubierto las **pruebas unitarias**. Utiliza la herramienta de cobertura de prueba para determinar si ha probado adecuadamente los elementos, clases, métodos y líneas de código que componen su aplicación.

Para esto tienes que hacer clic en **Ejecutar nombre-prueba con cobertura** 



Coverage: CalculatorTest x

33% classes, 55% lines covered in package 'com.example.myapplication...'

Element	Class, %	Method, %	Line, %
BuildConfig	0% (0/1)	0% (0/1)	0% (0/1)
Calculator	100% (1/1)	100% (5/5)	100% (5/5)
MainActivity	0% (0/1)	0% (0/2)	0% (0/3)

Porcentajes de cobertura de código para una aplicación.

Más pruebas unitarias

```
@Test
fun `username valido y password confirmada devuelve true`() {
    val result = UtilidadRegistro.validarInputRegistro(
        "Maria",
        "123",
        "123"
    )
    assertThat(result).isTrue()
}

@Test
fun `username existente devuelve false`() {
    val result = UtilidadRegistro.validarInputRegistro(
        "Alberto",
        "123",
        "123"
    )
    assertThat(result).isFalse()
}
```

```
@Test
fun `password incorrecta devuelve false`() {
    val result = UtilidadRegistro.validarInputRegistro(
        "Maria",
        "123456",
        "qwerty"
    )
    assertThat(result).isFalse()
}

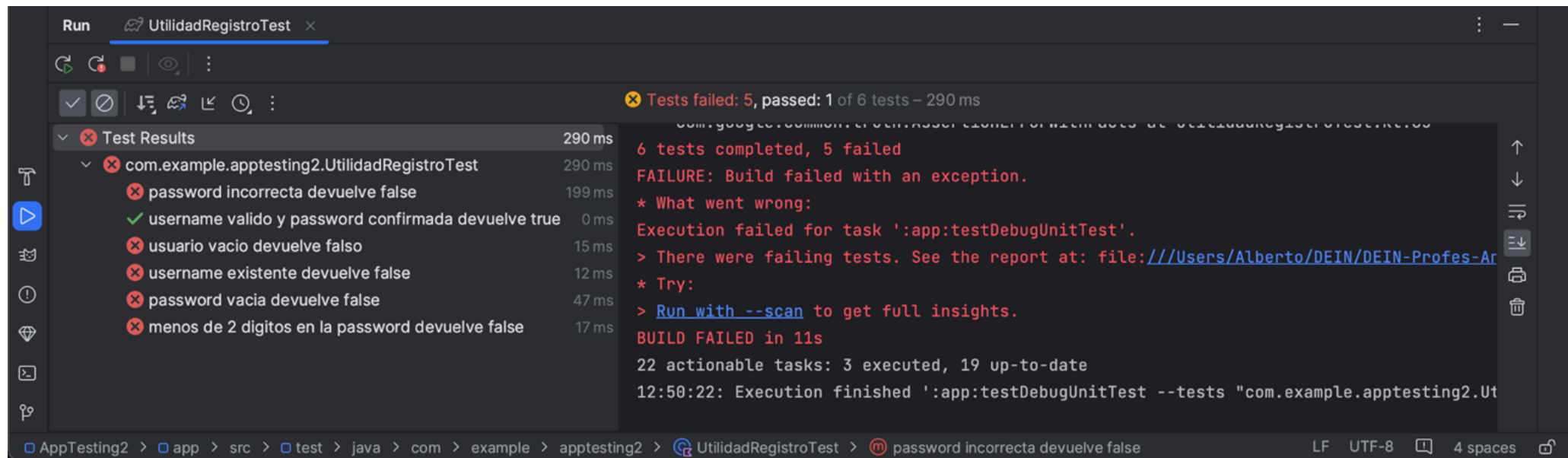
@Test
fun `password vacia devuelve false`() {
    val result = UtilidadRegistro.validarInputRegistro(
        "Maria",
        "",
        ""
    )
    assertThat(result).isFalse()
}

@Test
fun `menos de 2 digitos en la password devuelve false`() {
    val result = UtilidadRegistro.validarInputRegistro(
        "Maria",
        "qwerty1",
        "qwerty1"
    )
    assertThat(result).isFalse()
}
```

Resultado de las pruebas

Una vez realizadas las pruebas, siempre se realizarán en orden alfabético (puede ser distinto al orden de implementación en la clase de Test), podemos ver los resultados en la ventana de ejecución.

¿Por qué falla?



Hemos diseñado los casos de uso

Lo que está ocurriendo es que hemos hecho bien las cosas, pero **no hemos terminado**. Cuando se van a hacer pruebas, la programación ha de estar orientada a ellas. Lo que acabamos de hacer es definir los casos de uso de una función, que no hemos programado, pero que sí sabemos cómo queremos que funcione.

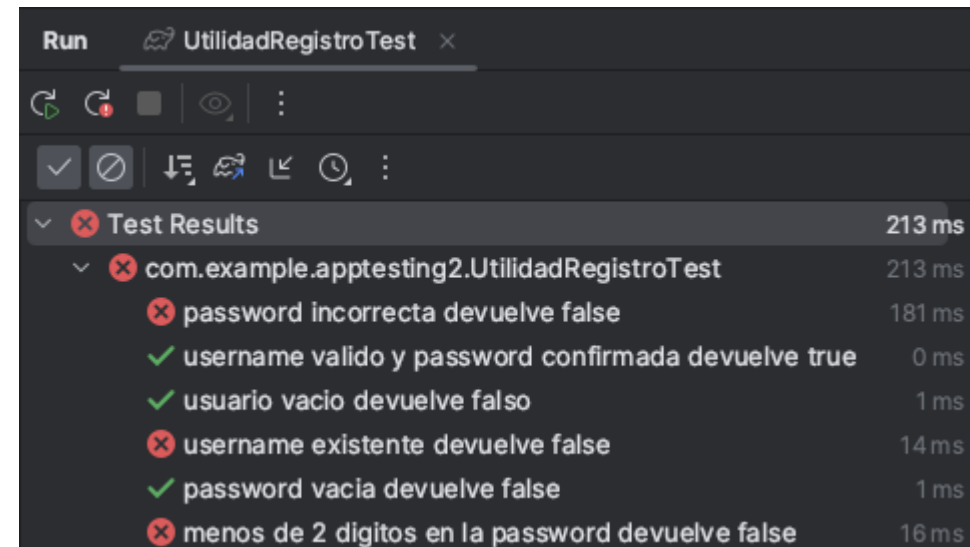
Ahora, el proceso es el siguiente:

- Vamos **programando la función**, y en el momento que tengamos alguna duda sobre si está bien programada, lanzamos la batería de pruebas unitarias.
- Los resultados de las pruebas nos dirán qué parte está mal programada.
- Una vez las pruebas unitarias nos den el OK, daremos por finalizada su programación.

Programando para pasar los test

En la primera versión de la función, siempre devolvíamos **True** y por eso la única prueba que pasaba el test, era la que se evaluaba como True. Ahora, vamos a añadir algo más de código y comprobaremos si el username o el password están vacíos.

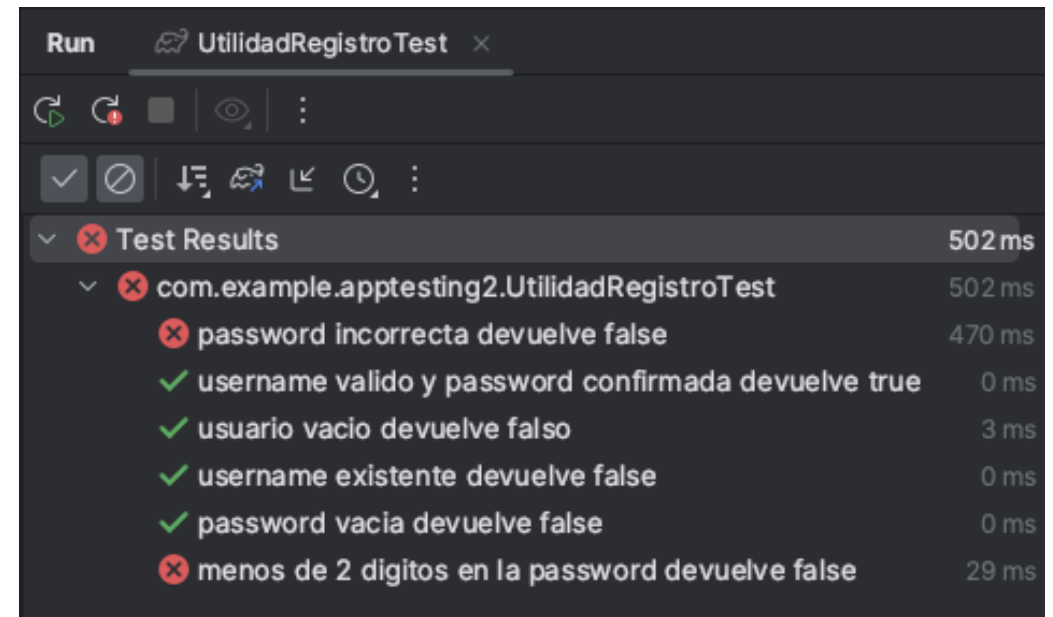
```
fun validarInputRegistro(
    username: String,
    password: String,
    confirmedPassword: String
): Boolean {
    if (username.isEmpty() || password.isEmpty()) {
        return false
    }
    else {
        return true
    }
}
```



Programando para pasar los test

Añadimos la comprobación para ver si el usuario existía previamente.

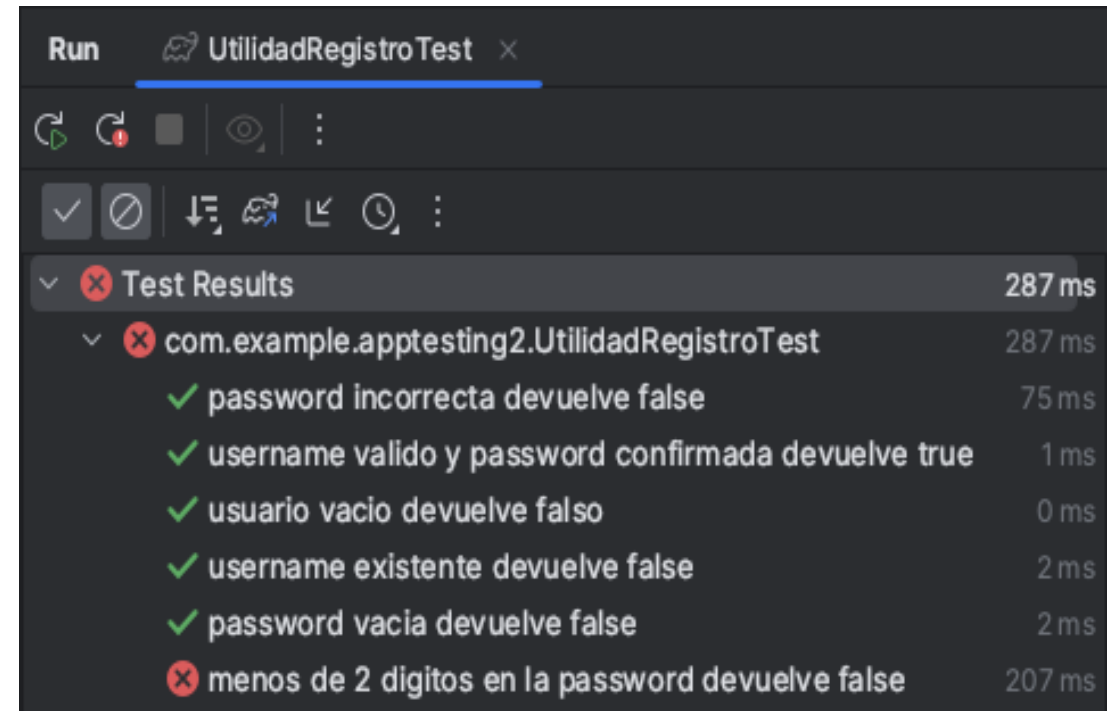
```
fun validarInputRegistro(  
    username: String,  
    password: String,  
    confirmedPassword: String  
) : Boolean {  
    if (username.isEmpty() ||  
        password.isEmpty()) {  
        return false  
    }  
    if (username in usuariosExistentes) {  
        return false  
    }  
    return true  
}
```



Programando para pasar los test

Añadimos la comprobación para ver si la confirmación y la password coinciden.

```
fun validarInputRegistro(
    username: String,
    password: String,
    confirmedPassword: String
): Boolean {
    if (username.isEmpty() || password.isEmpty()) {
        return false
    }
    if (username in usuariosExistentes) {
        return false
    }
    if (password != confirmedPassword) {
        return false
    }
    return true
}
```



Run UtilidadRegistroTest

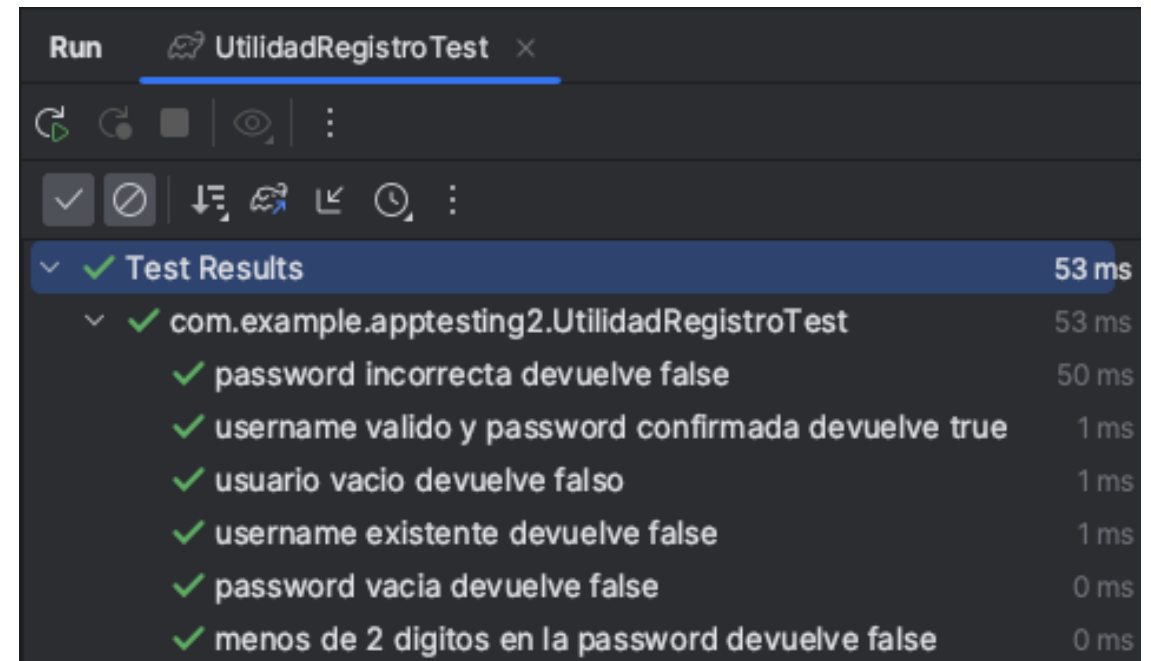
Test Results 287 ms

- com.example.apptesting2.UtilidadRegistroTest 287 ms
 - ✓ password incorrecta devuelve false 75 ms
 - ✓ username valido y password confirmada devuelve true 1 ms
 - ✓ usuario vacio devuelve falso 0 ms
 - ✓ username existente devuelve false 2 ms
 - ✓ password vacia devuelve false 2 ms
 - ✗ menos de 2 digitos en la password devuelve false 207 ms

Programando para pasar los test

Añadimos la comprobación para ver si la password contiene menos de 2 dígitos.

```
fun validarInputRegistro(
    username: String,
    password: String,
    confirmedPassword: String
): Boolean {
    if (username.isEmpty() || password.isEmpty()) {
        return false
    }
    if (username in usuariosExistentes) {
        return false
    }
    if (password != confirmedPassword) {
        return false
    }
    if (password.count { it.isDigit() } < 2) {
        return false
    }
    return true
}
```



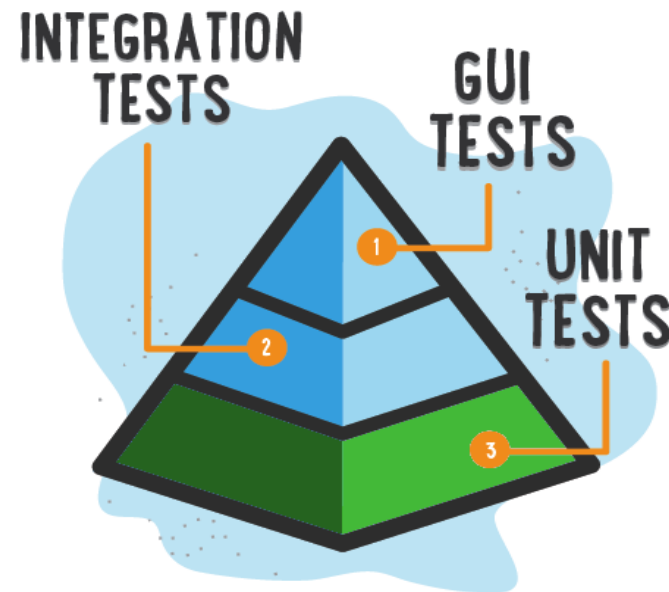
Simulación de dependencias de Android

Con la biblioteca **Mockable de Android** y los **frameworks ficticios**, como **Mockito** o **MockK**, puedes programar el comportamiento de las simulaciones de las clases de Android en tus pruebas de unidades.

Para **agregar un objeto ficticio** a tu prueba de unidad local con Mockito, sigue este modelo de programación:

1. Incluye la dependencia de **la biblioteca de Mockito** en el archivo **build.gradle**.
2. Al comienzo de la definición de la clase de prueba de unidades, agrega la anotación **@RunWith(MockitoJUnitRunner.class)**. Esa anotación le indica al ejecutor de pruebas de Mockito que valide que tu uso del framework sea correcto y simplifica la inicialización de tus objetos ficticios.
3. Si deseas crear un **objeto ficticio** para una dependencia de Android, agrega la anotación **@Mock** antes de la declaración del campo.
4. Para **indicar el comportamiento** de la dependencia, puedes especificar una condición y un valor que se muestre cuando se cumpla con la condición mediante los métodos **when()** y **thenReturn()**.

Pruebas de interfaz de usuario/UI, Test de instrumentación o Pruebas Instrumentales



Pruebas Instrumentadas o de integración

Estas pruebas se ejecutan en un dispositivo de **hardware o emulador**.

Tienen acceso a API que le brindan acceso a información, como la clase, en la aplicación que está probando y le permiten controlar la aplicación bajo prueba desde su código de prueba.

Las pruebas instrumentadas están integradas en un **APK independiente**, por lo que tienen su propio archivo. **Este archivo se genera automáticamente**, pero puedes crear tu propia versión en , que se fusionará con el manifiesto generado. Utilice pruebas instrumentadas al escribir pruebas de integración y de interfaz de usuario funcional para **automatizar la interacción del usuario**, o cuando sus pruebas tengan dependencias de Android para las que no pueda crear dobles de prueba.

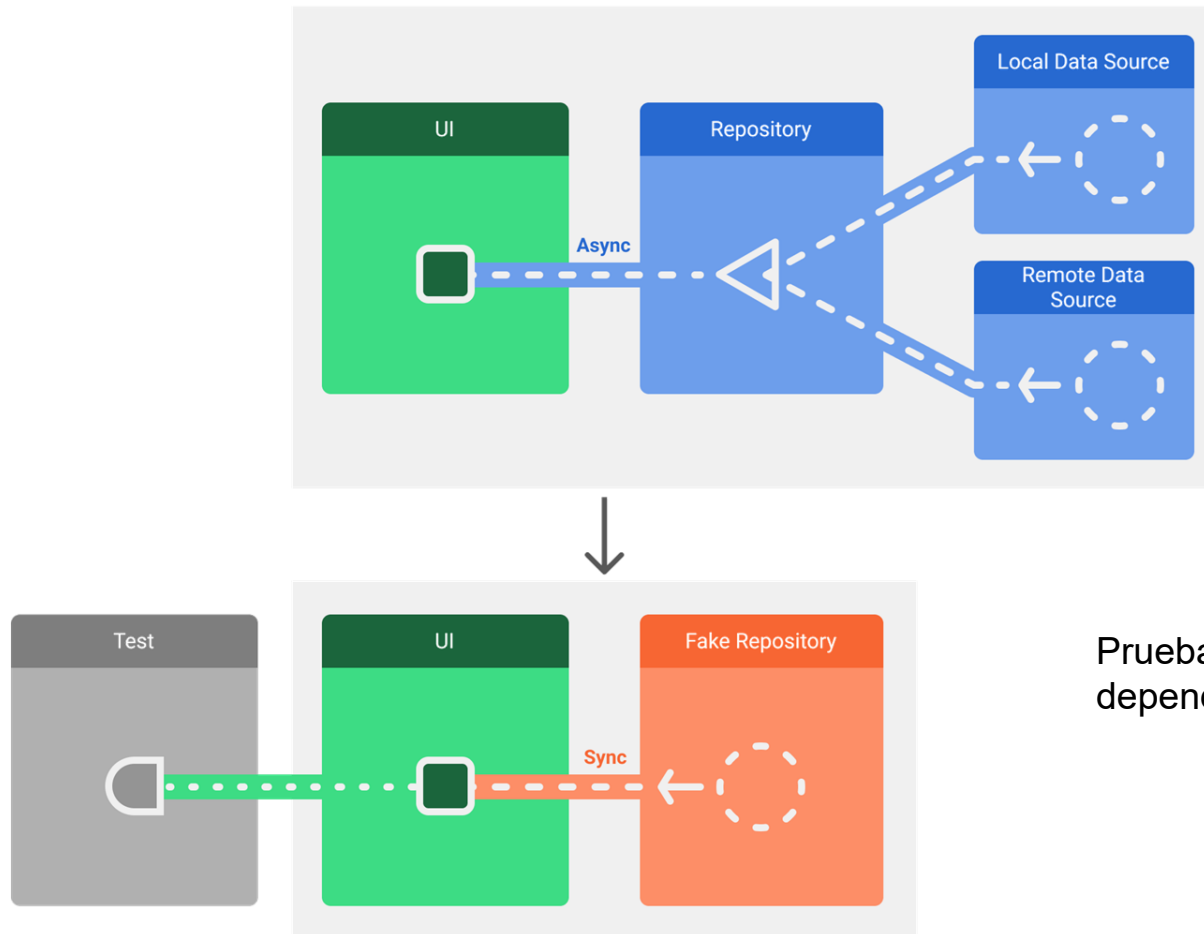
Probar las interacciones de los usuarios ayuda a garantizar que no encuentren resultados inesperados ni tengan una mala experiencia cuando interactúen con tu app. Debes adquirir el hábito de crear pruebas de interfaz de usuario (IU) si necesitas verificar que la IU de tu app funciona correctamente.

Pruebas Instrumentadas o de integración

Hay varios tipos de pruebas de IU que debes emplear:

- **Las pruebas de interfaz de usuario de pantalla** verifican las interacciones críticas del usuario en una sola pantalla. Realizan acciones como hacer clic en botones, escribir formularios y comprobar estados visibles. Una clase de prueba por pantalla es un buen punto de partida.
- **Pruebas de flujo de usuarios o pruebas de Navegación** , cubriendo los caminos más habituales. Estas pruebas simulan a un usuario moviéndose a través de un flujo de navegación. Son pruebas sencillas, útiles para comprobar si hay fallos en el tiempo de ejecución durante la inicialización.

Pruebas Instrumentadas o de integración



Prueba de una IU mediante el reemplazo de sus dependencias con emulaciones

Pruebas Instrumentadas o de integración

Un enfoque para las pruebas de la **Interfaz Grafica/UI** es simplemente hacer que un **verificador humano** realice un conjunto de operaciones del usuario en la app de destino y verifique que se comporte de forma correcta. Sin embargo, este enfoque manual puede llevar mucho tiempo y generar errores. Un enfoque más eficiente es **escribir las pruebas de la IU de modo que las acciones del usuario se realicen de manera automatizada**. El enfoque automatizado te permite ejecutar las pruebas de forma rápida y confiable de manera repetible.

Las pruebas de la IU inician una app (o parte de ella), **simulan las interacciones del usuario** y, por último, comprueban que la app reaccionó de forma adecuada. Son pruebas de integración que pueden variar desde la verificación del comportamiento de un componente pequeño hasta una prueba de navegación grande que recorre un flujo de usuarios completo. **Son útiles para comprobar la existencia de regresiones y la compatibilidad con diferentes niveles de API y dispositivos físicos.**

Pruebas Instrumentadas

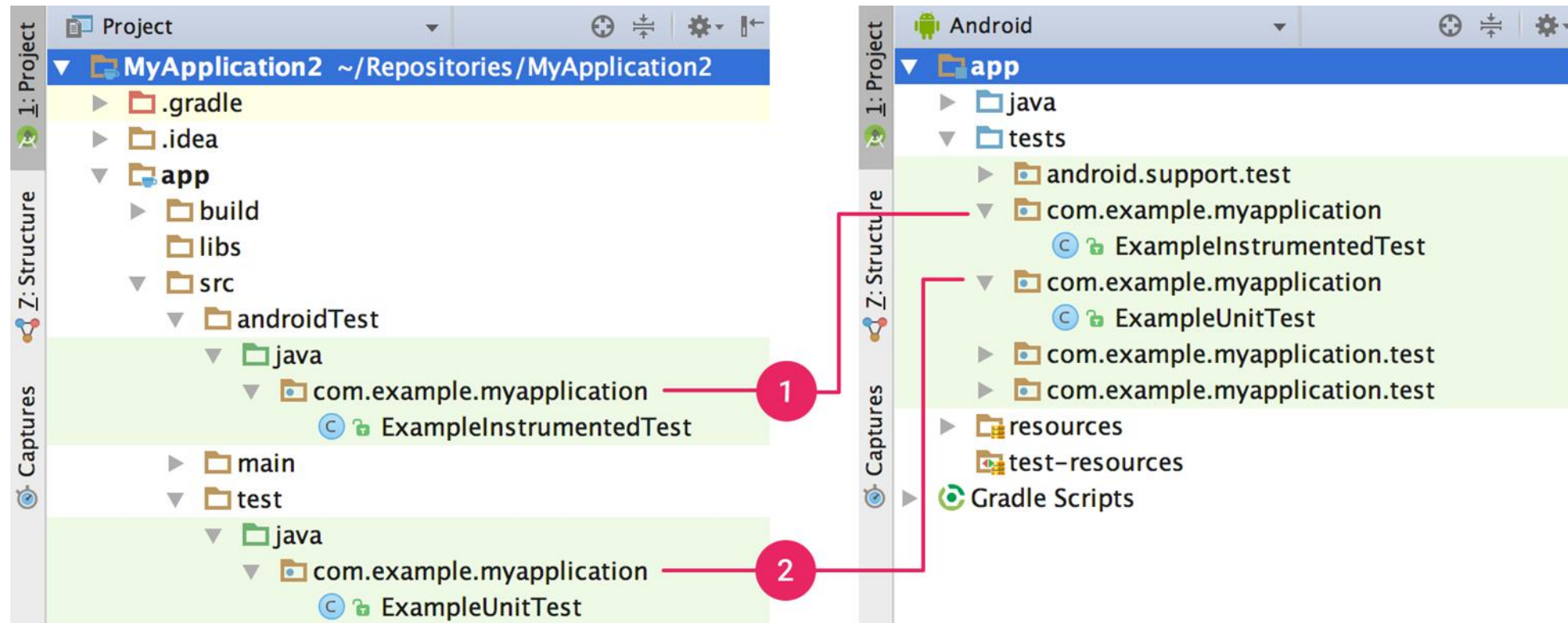
Para ejecutar pruebas de IU instrumentadas con Android Studio, **implementa el código de prueba** en una carpeta de pruebas de Android separada:

src/androidTest/java

El complemento de Android para **Gradle** compila una app de prueba **basada en el código de prueba** y, luego, carga la app de prueba en el mismo dispositivo que la app de destino. En tu código de prueba, puedes usar frameworks de prueba de IU para simular las interacciones de un usuario en la app de destino a fin de realizar tareas de prueba que abarcan situaciones de uso específicas.

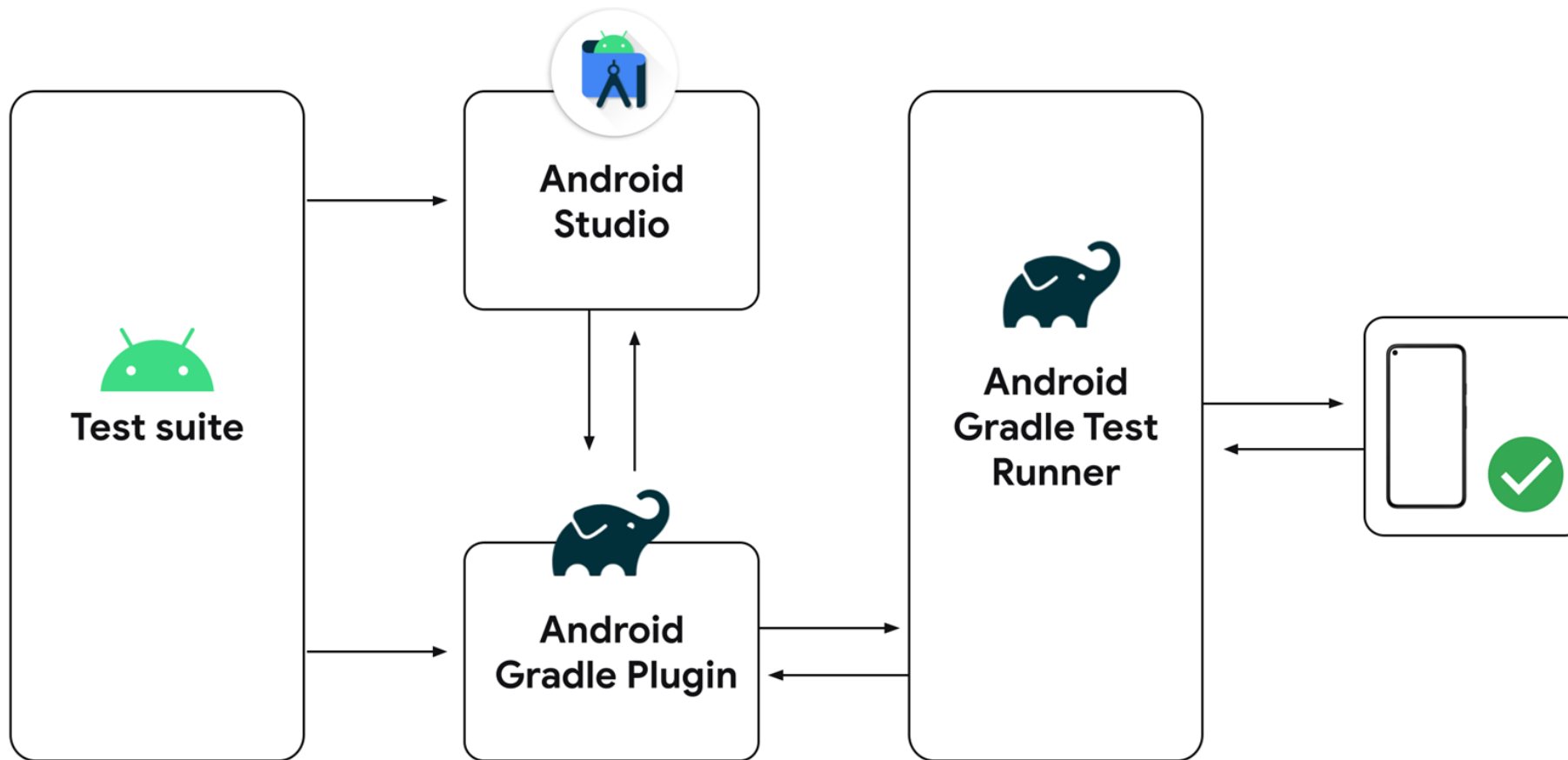


Pruebas Instrumentadas



Las (1) pruebas instrumentadas y (2) las pruebas JVM locales de su proyecto son visibles en la vista Proyecto (izquierda) o en la vista Android (derecha).

Pruebas Instrumentadas



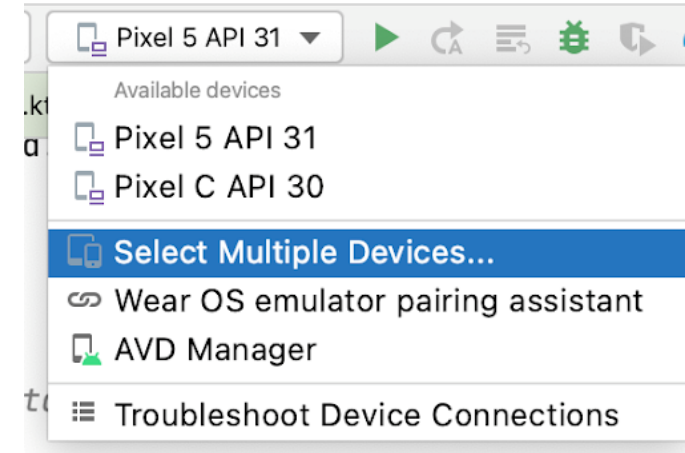
Ejecutor de pruebas unificado de Gradle.

Pruebas Instrumentadas

Las pruebas instrumentadas se ejecutan de forma predeterminada en un dispositivo físico o emulador.

Pero también se puede hacer en un conjunto más grande de dispositivos, seleccionar “más dispositivos”

La ventana de la herramienta Test Matrix muestra los resultados de la prueba para cada configuración de dispositivo seleccionada.

A screenshot of the 'Test Matrix' window in Android Studio. It shows the results of a test run for 'ExampleInstrumentedTest'. The table has columns for 'Tests', 'Duration', 'Status', 'Pixel_5_API_30', and 'Pixel_C_API_30'. The test 'useAppContext' passed on both devices.

Tests	Duration	Status	Pixel_5_API_30	Pixel_C_API_30
Test Results	207 ms	2/2	1/1	1/1
ExampleInstrumentedTest	207 ms	2/2	1/1	1/1
useAppContext	207 ms	Pass	✓	✓

Preparándonos para pruebas Instrumentadas

Según la [documentación de Android](#) tendremos que añadir varias dependencias al Gradle. [Versiones](#)

```
Groovy Kotlin
dependencies {
    // Core library
    androidTestImplementation("androidx.test:core:$androidXTestVersion")

    // AndroidJUnitRunner and JUnit Rules
    androidTestImplementation("androidx.test:runner:$testRunnerVersion")
    androidTestImplementation("androidx.test:rules:$testRulesVersion")

    // Assertions
    androidTestImplementation("androidx.test.ext:junit:$testJUnitVersion")
    androidTestImplementation("androidx.test.ext:truth:$truthVersion")

    // Espresso dependencies
    androidTestImplementation("androidx.test.espresso:espresso-core:$espressoVersion")
    androidTestImplementation("androidx.test.espresso:espresso-contrib:$espressoVersion")
    androidTestImplementation("androidx.test.espresso:espresso-intents:$espressoVersion")
    androidTestImplementation("androidx.test.espresso:espresso-accessibility:$espressoVersion")
    androidTestImplementation("androidx.test.espresso:espresso-web:$espressoVersion")
    androidTestImplementation("androidx.test.espresso.idling:idling-concurrent:$espressoVersion")

    // The following Espresso dependency can be either "implementation",
    // or "androidTestImplementation", depending on whether you want the
    // dependency to appear on your APK's compile classpath or the test APK
    // classpath.
    androidTestImplementation("androidx.test.espresso:espresso-idling-resource:$espressoVersion")
}
```

Artefacto	Versión estable
annotation	1.0.1
core	1.5.0
espresso	3.5.1
Dispositivo-espresso	-
ext.junit	1.1.5
ext.junit-gtest	-
ext.truth	1.5.0
monitor	1.6.1
orchestrator	1.4.2
runner	1.5.2
rules	1.5.0
services	1.4.2

Preparándonos para pruebas Instrumentadas

Según la [documentación de Android](#) tendremos que añadir varias dependencias al Gradle. [Versiones](#)

```
dependencies {  
  
    implementation("androidx.core:core-ktx:1.12.0")  
    implementation("androidx.appcompat:appcompat:1.6.1")  
    implementation("com.google.android.material:material:1.11.0")  
    implementation("androidx.constraintlayout:constraintlayout:2.1.4")  
  
    testImplementation("junit:junit:4.13.2")  
  
    androidTestImplementation("androidx.test.espresso:espresso-core:3.5.1")  
  
    androidTestImplementation("androidx.test:runner:1.5.2")  
    androidTestImplementation("androidx.test:core:1.5.0")  
  
    androidTestImplementation("androidx.test.ext:junit:1.1.5")  
}
```

Crear un proyecto de prueba

Vamos a crear un proyecto simple con dos Activities y en cada una, habrá un título y un botón.

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:id="@+id/main">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        android:id="@+id/activity_main_title"
        android:text="@string/text_mainactivity"
        android:textSize="25sp"
        android:textColor="#000"
        android:layout_marginTop="80dp" />

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/button_next_activity"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/activity_main_title"
        android:layout_marginTop="50dp"
        android:text="@string/siguiente" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:id="@+id/secondary">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        android:id="@+id/activity_secondary_title"
        android:text="@string/segunda_activity"
        android:textSize="25sp"
        android:textColor="#000"
        android:layout_marginTop="80dp" />

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/button_back"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/activity_secondary_title"
        android:layout_marginTop="50dp"
        android:text="@string/boton_atras" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Crear un proyecto de prueba

Haremos que se pueda ir de una activity a otra navegando con los botones.

```
package com.example.apptesting1

import android.content.Intent
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val button_next_activity: Button = findViewById(R.id.button_next_activity)

        button_next_activity.setOnClickListener {
            val intent = Intent(this, SecondaryActivity::class.java)
            startActivity(intent)
        }
    }
}
```

```
package com.example.apptesting1

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button

class SecondaryActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_secondary)

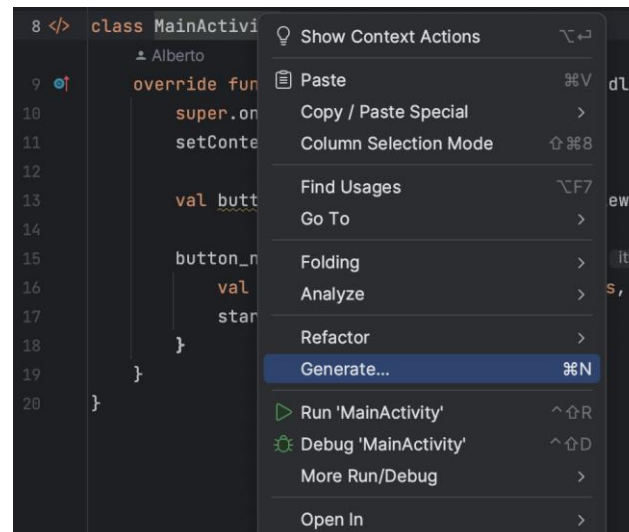
        val button_back: Button = findViewById(R.id.button_back)

        button_back.setOnClickListener {
            onBackPressedDispatcher.onBackPressed()
        }
    }
}
```


Crear nuevas pruebas Instrumentadas

Para agregar una nueva prueba para una clase o método específico directamente desde su código fuente sigue estos pasos:

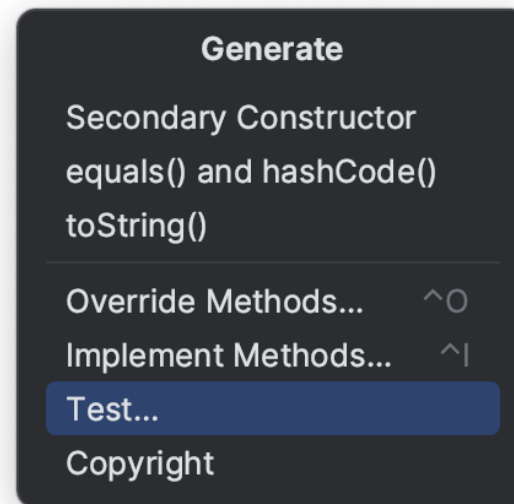
1. **Abre el archivo** fuente que contiene el código que desea probar. (ej. MainActivity)
2. Coloca el cursor en el nombre de la **clase o método que desea probar** y presiona **Control+Shift+T** (⌘+N macOS).



Crear nuevas pruebas Instrumentadas

Para agregar una nueva prueba para una clase o método específico directamente desde su código fuente sigue estos pasos:

3. En la ventana emergente que aparece, haz clic en **Test...**

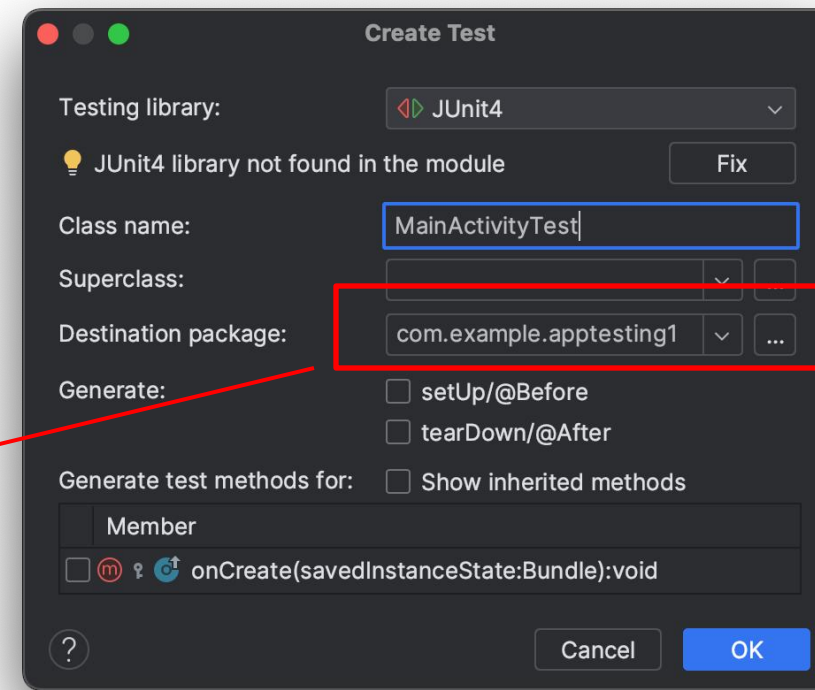
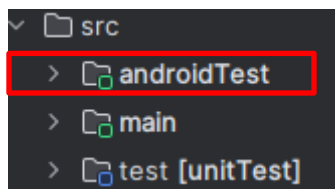


Crear nuevas pruebas Instrumentadas

Para agregar una nueva prueba para una clase o método específico directamente desde su código fuente sigue estos pasos:

4. En el cuadro de diálogo Crear prueba , elija **JUnit4** y luego haga clic en Aceptar.

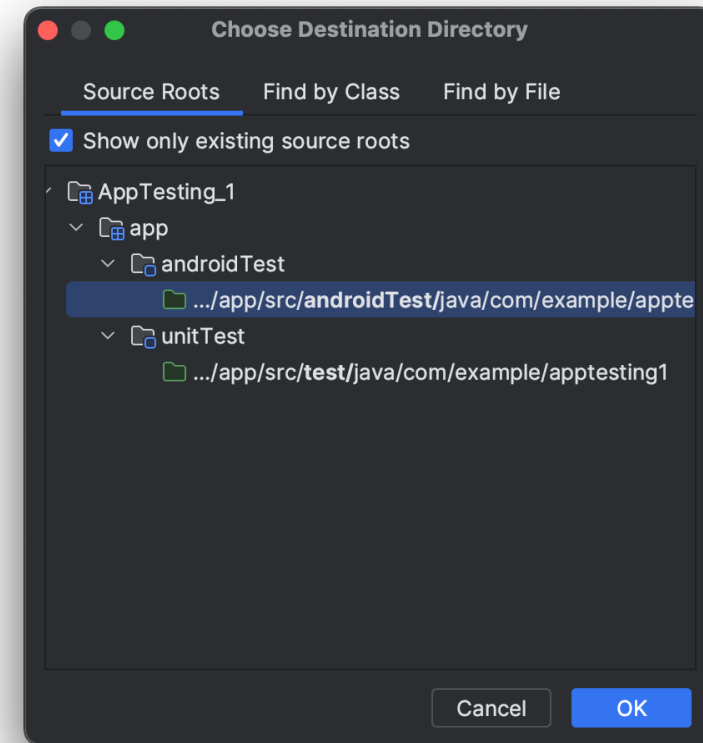
Ten cuidado en la carpeta donde se crea:



Crear nuevas pruebas Instrumentadas

Para agregar una nueva prueba para una clase o método específico directamente desde su código fuente sigue estos pasos:

5. En el cuadro de diálogo Elegir directorio de destino , haz clic en el conjunto de fuentes correspondiente al tipo de prueba que desea crear: **“androidTest”** para una prueba instrumentada o **“unitTest”** para una prueba de unidad local (unitaria). Luego haz clic en **Aceptar** .



Crear nuevas pruebas Instrumentadas

Alternativamente, puede crear un archivo de prueba genérico en el conjunto de fuentes de prueba apropiado de la siguiente manera:

1. En la ventana del **Proyecto** a la izquierda, haga clic en el menú desplegable y seleccione la **vista de Android**.
2. Haga clic derecho en el directorio **java** y seleccione **Nuevo > Clase/Archivo Kotlin**.
Alternativamente, puede seleccionar el directorio java y usar el acceso directo Control+N(Command+N en macOS).
3. En el cuadro de diálogo Elegir directorio de destino, haga clic en el conjunto de fuentes correspondiente al tipo de prueba que desea crear: **“androidTest”** para una prueba instrumentada o **“unitTest”** para una prueba de unidad local (unitaria). Luego haga clic en Aceptar.
4. Asigne un nombre al archivo y luego haga clic en Aceptar.

Primera prueba, ¿Se abre la MainActivity?

En la nueva clase creada, **MainActivityTest**, vamos a crear nuestra primera prueba instrumentada.

```
import androidx.test.internal.runner.junit4.AndroidJUnit4ClassRunner
import org.junit.runner.RunWith

@RunWith(AndroidJUnit4ClassRunner::class)
class MainActivityTest{

    //Carga Activity
    @Test
    fun test_MainActivity_carga() {
        val activityScenario = ActivityScenario.launch(MainActivity::class.java)

        onView(withId(R.id.main)).check(matches(isDisplayed()))
    }
}
```

Primera prueba, ¿Se abre la MainActivity?

Utilizando la palabra reservada **@Test**, indicamos a la clase de pruebas, que vamos a definir una prueba.

Una de las reglas cuando estamos realizando pruebas, es **que los nombres tienen que ser lo más descriptivos posible**. No importa que el nombre de la función sea muy largo, lo importante es que quede muy claro qué prueba esa función.

```
fun test_MainActivity_carga() {
```

Para realizar las pruebas, necesitamos que se lance el MainActivity, igual que cuando probamos nuestra app con el emulador de Android Studio. Para ello, en las clases de prueba, necesitamos generar un escenario con la activity que queremos lanzar.

```
val activityScenario = ActivityScenario.launch(MainActivity::class.java)
```

Primera prueba, ¿Se abre la MainActivity?

Una vez está lanzada la MainActivity, vamos a comprobar si esta, **se carga correctamente y se visualiza**. Para comprobar esto, tenemos que utilizar el método **onView**. Con este método podremos acceder a todas las vistas que se encuentren a la vista. En este caso, estamos accediendo a todo el layout. El acceso al layout se hace igual que accedemos a otras vistas, desde el **R.id**.

```
onView(withId(R.id.main)).check(matches(isDisplayed()))
```

Fijaros que en la definición del **activity_main.xml** también le hemos dado un id al layout.

```
<androidx.constraintlayout.widget.ConstraintLayout  
  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    xmlns:app="http://schemas.android.com/apk/res-auto"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    android:id="@+id/main">
```


Ejecutar pruebas

Primero, asegúrate de que tu proyecto esté completamente sincronizado con **Gradle**. 

- Para **ejecutar todas las pruebas en un directorio o archivo**, abra la ventana **Proyecto** y realice una de las siguientes acciones:
 - ☐ Haz clic derecho en un directorio o archivo y haga clic en **Run/Ejecutar**. 
 - ☐ Selecciona el directorio o archivo y use el acceso directo Control+Shift+R.

Para **ejecutar todas las pruebas en una clase o un método específico**, abra el archivo de prueba en el Editor de código y realice una de las siguientes acciones:

- ☐ Presiona el ícono **Ejecutar prueba** en el **gutter/canal**. 
- ☐ Haz clic derecho en la clase o método de prueba y haga clic en **Run/Ejecutar** 
- ☐ Selecciona la clase o método de prueba y use el acceso directo Control+Shift+R.



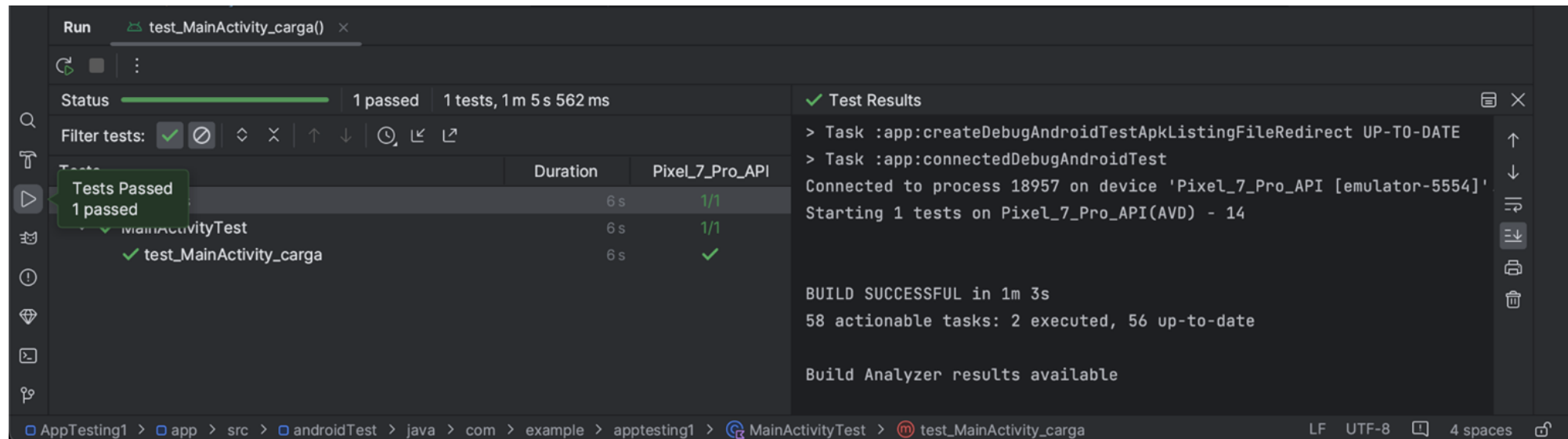
```
1 package com.  
2  
3 > import ...  
14  
15 @RunWith(And  
16 > class MainAc  
17  
18 //Carga  
19  
20 > @Test  
21 fun test  
22 val  
23  
24 onVi  
}
```

Sus pruebas instrumentadas se ejecutarán en un dispositivo físico o emulador.

Resultado de las pruebas

Una vez realizadas las pruebas, siempre se realizarán en orden alfabético (puede ser distinto al orden de implementación en la clase de Test), podemos ver los resultados en la ventana de ejecución.

Podemos ver en el emulador como la aplicación se abre y se cierra automáticamente, y simplemente realiza la prueba que hemos programado.



Segunda prueba, ¿Se ven las vistas?

En la nueva clase creada, **MainActivityTest**, vamos a crear una prueba de **visibilidad** de forma que una vez se abre la MainActivity, verifique si el título y el botón están visibles.

```
// Visibilidad
@Test
fun test_Visibilidad_titulo_y_nextButton() {
    val activityScenario = ActivityScenario.launch(MainActivity::class.java)

    onView(withId(R.id.activity_main_title))
        .check(matches(isDisplayed()))

    onView(withId(R.id.button_next_activity))
        .check(matches(isDisplayed()))
}
```

Tercera prueba, ¿Qué pone en el título?

En la nueva clase creada, **MainActivityTest**, vamos a crear una prueba **de texto** para que una vez se abre la MainActivity, verifique si el título tiene escrito un determinado String.

```
// Texto
@Test
fun test_Texto_titulo_mostrado() {
    val activityScenario = ActivityScenario.launch(MainActivity::class.java)

    onView(withId(R.id.activity_main_title))
        .check(matches(withText(R.string.text_mainactivity)))
}
```

Cuarta prueba, ¿Funciona la navegación?

En la nueva clase creada, **MainActivityTest**, vamos a crear una prueba **navegación** entre activities para que una vez se abre la MainActivity, verifique si haciendo click en el botón, nos lleva a la activity correcta. Para realizar la acción de click, tenemos el método perform, que recibe una [ViewAction](#).

```
// Navegación
@Test
fun test_Navegacion_a_SecondaryActivity() {

    val activityScenario = ActivityScenario.launch(MainActivity::class.java)

    onView(withId(R.id.button_next_activity)).perform(click())
    onView(withId(R.id.secondary)).check(matches(isDisplayed()))
}
```

Quinta prueba, ¿Funciona la navegación de verdad?

En la nueva clase creada, **MainActivityTest**, vamos a crear una prueba **navegación** entre activities similar a la anterior, pero comprobando que una vez estemos en la segunda activity, podemos darle al botón y volver de nuevo a la MainActivity.

```
// Navegación ida y vuelta entre SecondaryActivity to MainActivity
@Test
fun test_Navegacion_de_MainActivity_a_SecondaryActivity_y_vuelta() {

    val activityScenario = ActivityScenario.launch(MainActivity::class.java)

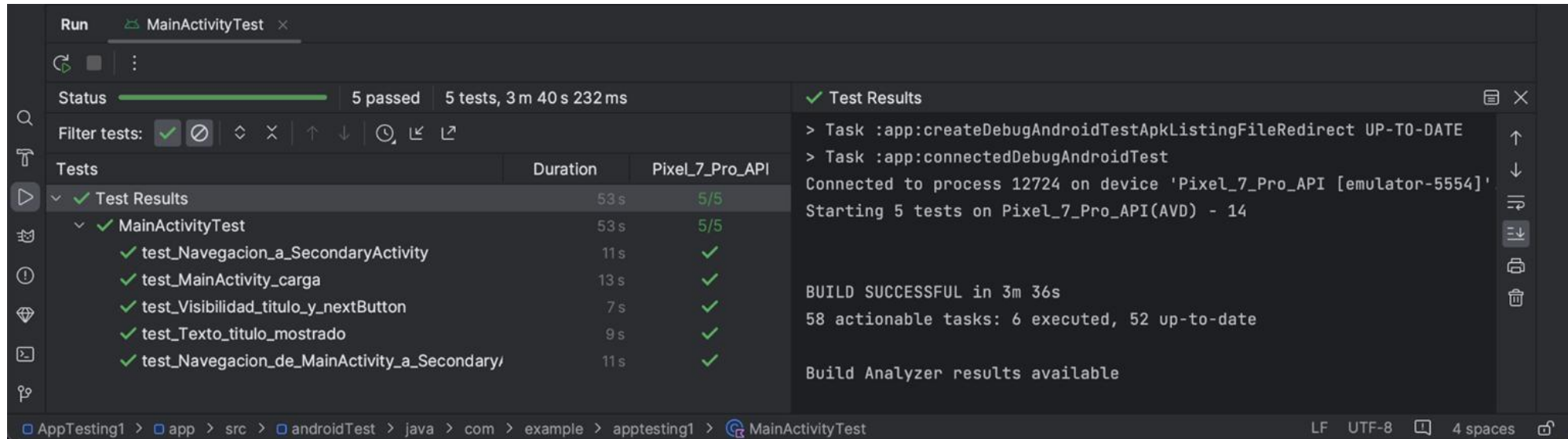
    onView(withId(R.id.button_next_activity)).perform(click())

    onView(withId(R.id.secondary)).check(matches(isDisplayed()))
    onView(withId(R.id.button_back)).perform(click()) // method 1

    onView(withId(R.id.main)).check(matches(isDisplayed()))
    onView(withId(R.id.button_next_activity)).perform(click())
}
```

Resultado de las 5 pruebas

Podemos realizar cada prueba por separado, o todas las pruebas de una clase. Además, si tenemos varias clases en la carpeta **androidTest**, podemos realizar todas las pruebas de todas las clases que se encuentran en su interior.



The screenshot shows the Android Studio interface with the 'Run' tab selected. The 'Test Results' section displays a table of test results for the 'MainActivityTest' class. The table has three columns: 'Tests', 'Duration', and 'Pixel_7_Pro_API'. The results show 5 tests passed, all with a duration of 11s or less, and all marked as '5/5' on the 'Pixel_7_Pro_API' device. The 'Test Results' section also shows the 'Test Results' status as '5 passed' and '5 tests, 3 m 40 s 232 ms'.

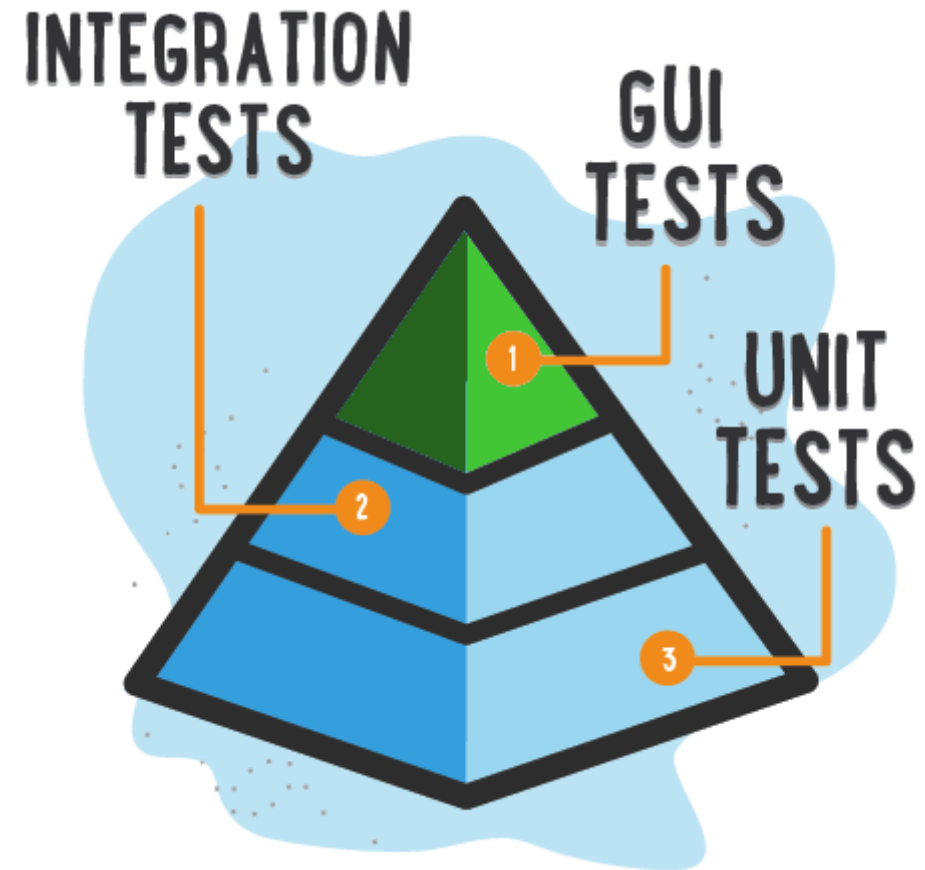
Tests	Duration	Pixel_7_Pro_API
✓ Test Results	53 s	5/5
✓ MainActivityTest	53 s	5/5
✓ test_Navegacion_a_SecondaryActivity	11 s	✓
✓ test_MainActivity_carga	13 s	✓
✓ test_Visibilidad_titulo_y_nextButton	7 s	✓
✓ test_Texto_titulo_mostrado	9 s	✓
✓ test_Navegacion_de_MainActivity_a_Secondary/	11 s	✓

The 'Test Results' section also shows the following build output:

```
> Task :app:createDebugAndroidTestApkListingFileRedirect UP-TO-DATE
> Task :app:connectedDebugAndroidTest
Connected to process 12724 on device 'Pixel_7_Pro_API [emulator-5554]'
Starting 5 tests on Pixel_7_Pro_API(AVD) - 14

BUILD SUCCESSFUL in 3m 36s
58 actionable tasks: 6 executed, 52 up-to-date

Build Analyzer results available
```



Grabar pruebas con Espresso

La herramienta **Espresso Test Recorder** te permite crear pruebas de interfaz de usuario para tu aplicación sin escribir ningún código de prueba. Al grabar un escenario de prueba, puedes registrar tus interacciones con un dispositivo y agregar afirmaciones para verificar elementos de la interfaz de usuario en instantáneas particulares de tu aplicación. Luego, Espresso Test Recorder **toma la grabación guardada y genera automáticamente una prueba de interfaz de usuario** correspondiente que puedes ejecutar para probar tu aplicación.

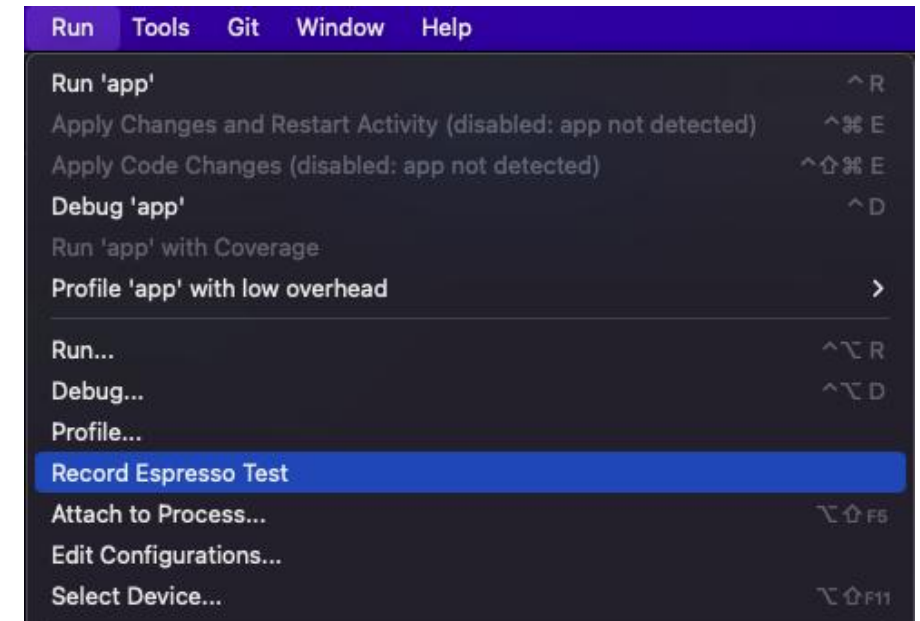
Las pruebas de Espresso consisten en dos componentes principales: **interacciones de interfaz de usuario y afirmaciones sobre elementos de vista**. Las interacciones de interfaz de usuario incluyen acciones de tocar y escribir que una persona puede usar para interactuar con tu aplicación. Las afirmaciones verifican la existencia o el contenido de elementos visuales en la pantalla.



Grabar pruebas con Espresso

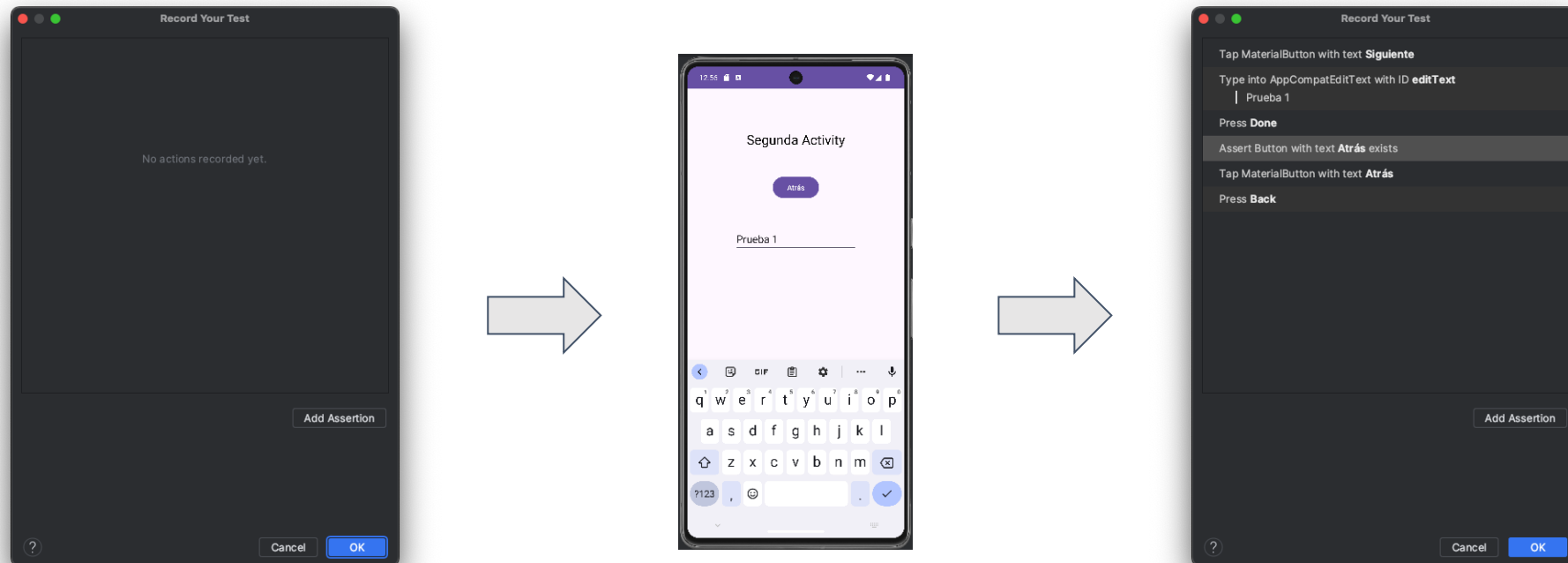
Para comenzar a grabar una prueba con Espresso Test Recorder, sigue estos pasos:

1. Haz clic en **Run > Record Espresso Test**
2. Espresso Test Recorder desencadena una compilación de tu proyecto y la aplicación debe instalarse y lanzarse antes de que Espresso Test Recorder te permita interactuar con ella.



Grabar pruebas con Espresso

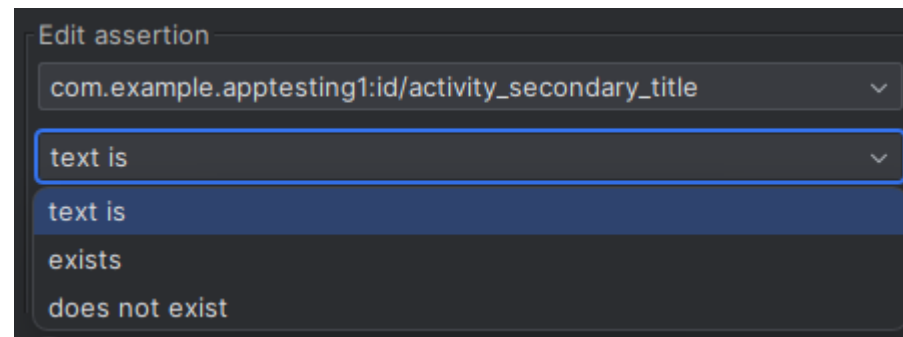
La ventana **Grabar Tu Prueba** aparece después de que se lance la aplicación y, como aún no has interactuado con el dispositivo, el panel principal muestra "Aún no se han registrado eventos". Interactúa con tu dispositivo para comenzar a registrar eventos como acciones de "tocar" y "escribir".



Agrega afirmaciones para verificar elementos

Las **afirmaciones/assertion** verifican la existencia o el contenido de un elemento de vista a través de tres tipos principales:

- **text is:** Verifica el contenido de texto del elemento de vista seleccionado.
- **exists:** Verifica que el elemento de vista esté presente en la jerarquía de vista actual visible en la pantalla.
- **does not exist:** Verifica que el elemento de vista no esté presente en la jerarquía de vista actual.



Agrega afirmaciones para verificar elementos

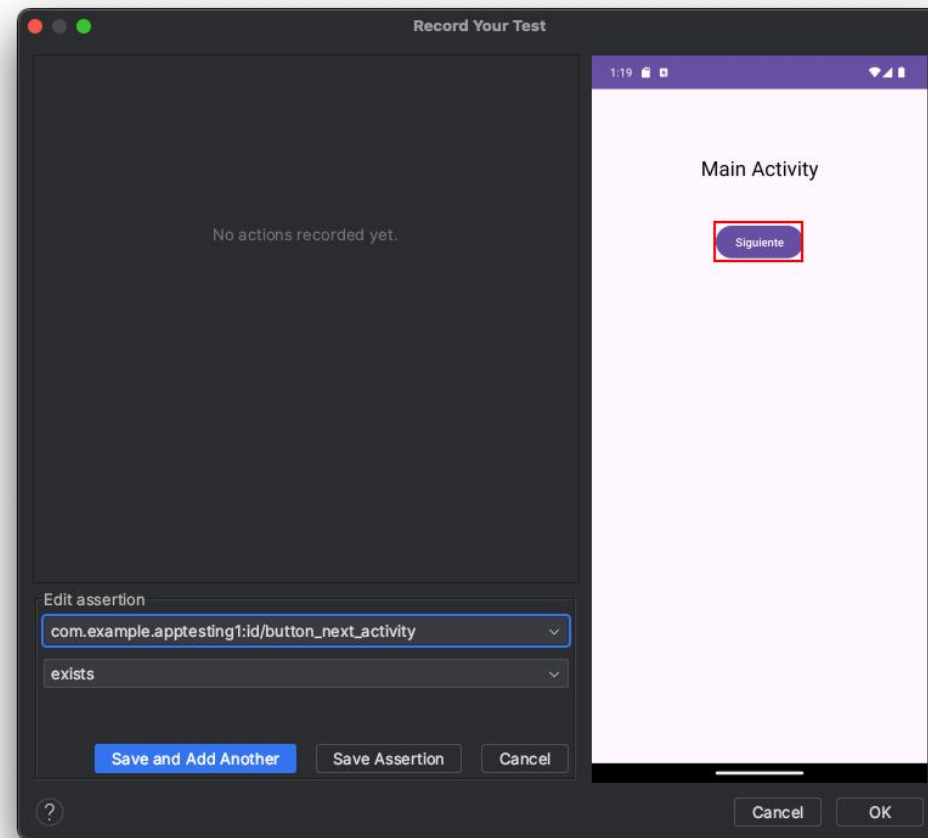
Para agregar una afirmación a tu prueba, sigue estos pasos:

1. Haz clic en **Agregar Afirmación**. Aparecerá un cuadro de diálogo **Captura de Pantalla** mientras **Espresso** obtiene la jerarquía de la interfaz de usuario y otra información sobre el estado actual de la aplicación. El cuadro de diálogo se cerrará automáticamente una vez que Espresso haya capturado la captura de pantalla.
2. Se mostrará un **diseño de la pantalla actual** en un panel a la derecha de la ventana **Grabar Tu Prueba**. Para seleccionar un elemento de vista en el que crear una afirmación, haz clic en el elemento en la captura de pantalla o utiliza el primer menú desplegable en el cuadro de edición de la afirmación en la parte inferior de la ventana. El objeto de vista seleccionado se resaltarán en un **cuadro rojo**.

Agrega afirmaciones para verificar elementos

3. **Selecciona la afirmación** que deseas usar en el segundo menú desplegable en el cuadro de edición de la afirmación. **Espresso** poblará el menú con afirmaciones válidas para el elemento de vista seleccionado.
4. Si eliges la afirmación "**el texto es**", Espresso insertará automáticamente el texto que se encuentra actualmente dentro del elemento de vista seleccionado. Puedes editar el texto para que coincida con tu afirmación deseada utilizando el campo de texto en el cuadro de edición de la afirmación.
5. Haz clic en **Guardar y Agregar Otro** para crear otra afirmación o haz clic en **Guardar Afirmación** para cerrar los paneles de afirmación.

Agrega afirmaciones para verificar elementos



Guardar la grabación

Haz clic en **Completar Grabación**. Aparecerá la ventana “**Elegir un nombre de clase de prueba**” para tu prueba.

El archivo se abre automáticamente después de que **Espresso Test Recorder** lo genere, y Android Studio muestra la clase de prueba como seleccionada en la ventana Proyecto del IDE.

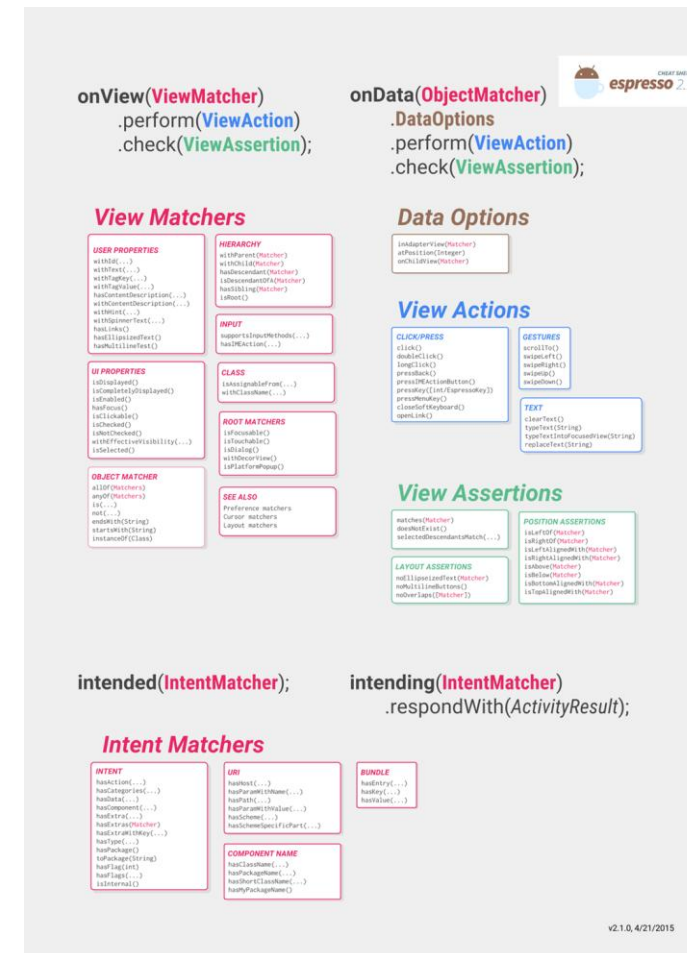
Dónde se guarda la prueba depende de la ubicación de tu raíz de **prueba de instrumentación**, así como del nombre del paquete de la actividad lanzada.

Por ejemplo, las pruebas para nuestra aplicación se guardan en la carpeta `src > androidTest > java > com.example.apptesting1` del módulo de la aplicación en el que grabaste la prueba.

Ejecutar una prueba de Espresso localmente

Estas pruebas se ejecutan de la misma manera que las pruebas instrumentales programadas.

Una de las ventajas que tienen es **que el código que generan es modificable**, y nos permite hacer una interacción completa de manera gráfica para posteriormente, por ejemplo, añadir bucles que rellenen campos con listas de valores conflictivos, realizar un scroll durante 1 minuto, o forzar la aplicación a clicar en todos y cada uno de los objetos de un recyclerView y comprobar que todos funcionan.



Ejecutar pruebas en Firebase



Firestore Test Lab te permite probar tu aplicación en una variedad de dispositivos y configuraciones.

Cuando ejecuta una prueba o un conjunto de casos de prueba en dispositivos y configuraciones que ha seleccionado, Test Lab ejecuta la prueba en su aplicación en un lote y luego muestra los resultados como una matriz de prueba .

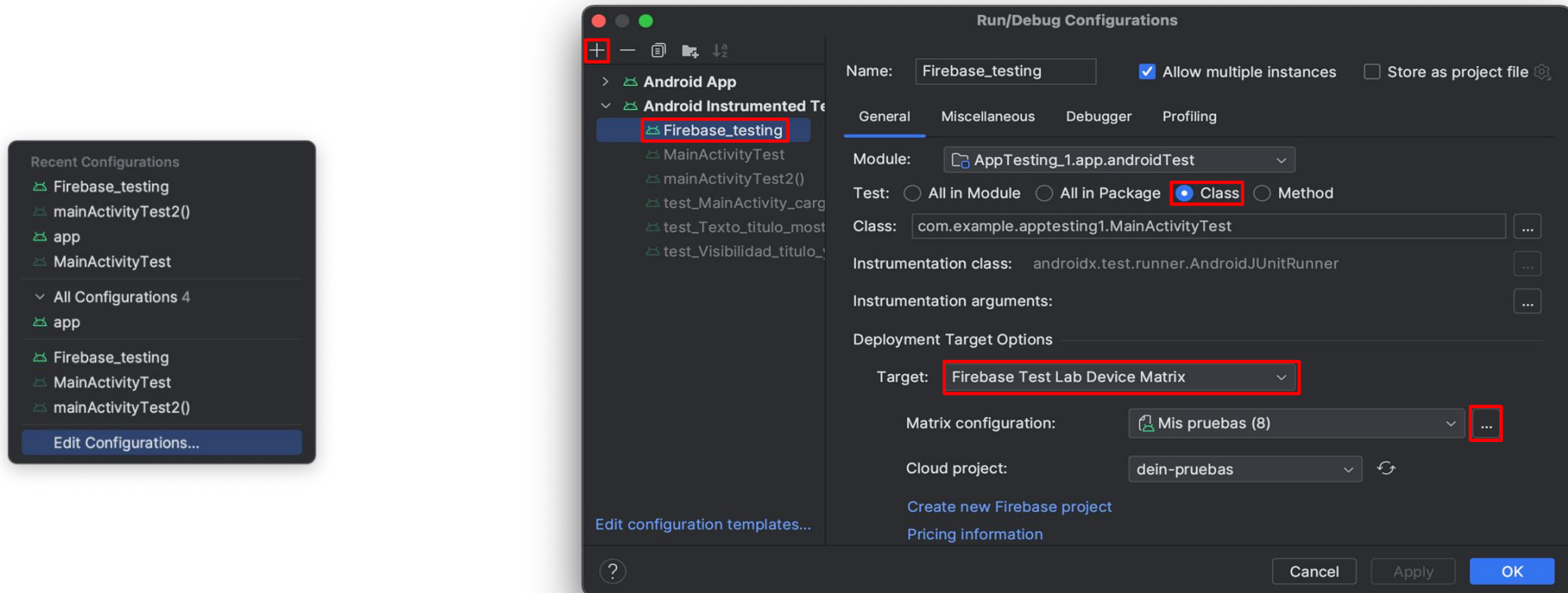
Dispositivos × Ejecuciones de prueba = Matriz de prueba

- Un dispositivo físico o virtual (solo Android) en el que realiza una prueba, como un teléfono, una tableta o un dispositivo portátil.
- Una prueba (o un conjunto de casos de prueba) que se ejecutará en un dispositivo.
- La matriz contiene los estados y resultados de las pruebas para sus ejecuciones de pruebas.

Ejecutar pruebas en Firebase



Primero creamos la configuración de ejecución de pruebas:

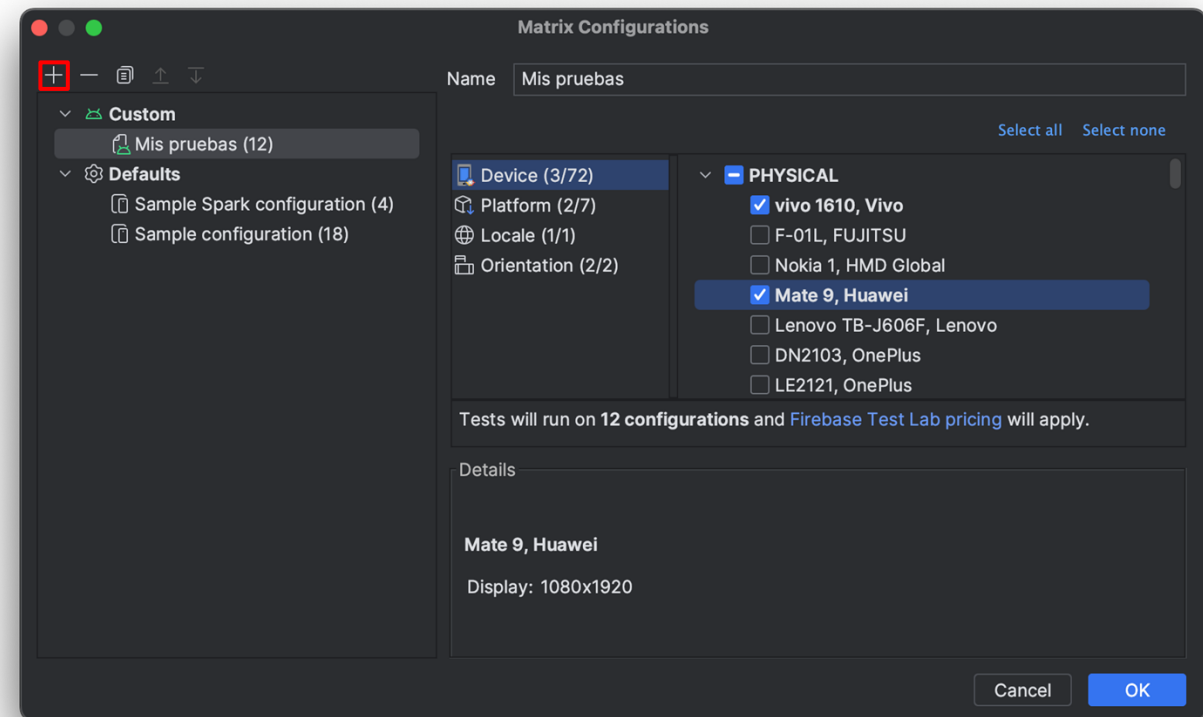


Ejecutar pruebas en Firebase



Creamos la matriz de configuración de dispositivos

1. Podemos elegir entre dispositivos físicos o virtuales. (diferentes precios)
2. Diferentes plataformas.
 1. Diferentes idiomas.
 2. Diferentes orientaciones.



Ejecutar pruebas en Firebase



Arrancamos las pruebas en nuestro Android Studio que está conectado con los **servidores de Google**. Esto es muy cómodo y te permite probar todas tus pruebas en todos los dispositivos que quieras, con los sistemas operativos android que quieras, en los idiomas que quieras y en la orientación que quieras.

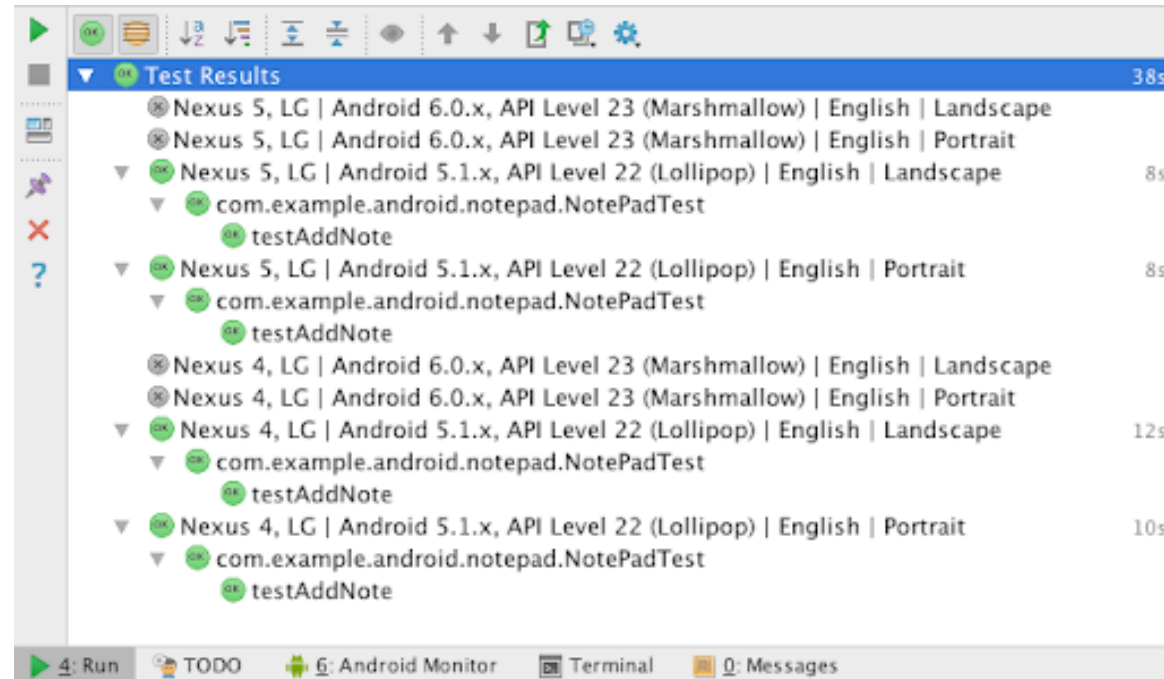
Google nos deja realizar **10 test por día** en dispositivos virtuales y **5 test** en dispositivos físicos. Para todo lo demás [Mastercard](#).

```
Galaxy S22 Ultra, Samsung | Android 14, API Level 34 (UpsideDownCake) | Spanish | Landscape
Galaxy S22 Ultra, Samsung | Android 14, API Level 34 (UpsideDownCake) | Spanish | Portrait
Galaxy S9, Samsung | Android 14, API Level 34 (UpsideDownCake) | Spanish | Landscape
Galaxy S9, Samsung | Android 14, API Level 34 (UpsideDownCake) | Spanish | Portrait
Galaxy S9, Samsung | Android 13, API Level 33 (T) | Spanish | Landscape
Galaxy S9, Samsung | Android 13, API Level 33 (T) | Spanish | Portrait
```

Resultados de las pruebas



Cuando Firebase Test Lab complete la ejecución de sus pruebas, se abrirá la **ventana Ejecutar para mostrar los resultados**:



También puede analizar sus pruebas en la web siguiendo el enlace que se muestra al comienzo del registro de ejecución de la prueba en la ventana Ejecutar .

Cómo compilar pruebas de unidades locales

Una prueba local se ejecuta directamente en el programa en lugar de un dispositivo o emulador de Android. Las pruebas locales te permiten evaluar la lógica de tu app más rápido. Sin embargo, no poder interactuar con el framework de Android crea una limitación en los tipos de pruebas que puedes ejecutar.

Una **prueba de unit** verifica el comportamiento de una pequeña sección de código, la unidad que se está probando. Para ello, ejecuta el código y verifica el resultado. Suelen ser simples, pero su configuración puede causar problemas cuando la unidad que se está probando no se diseñó teniendo en cuenta la capacidad de prueba, por ejemplo, **el código que deseas verificar debe ser accesible desde una prueba** (no puedes probar un método privado de forma directa, en su lugar, debes probar la clase con sus API públicas).

Ejecutar con Firebase Test Lab

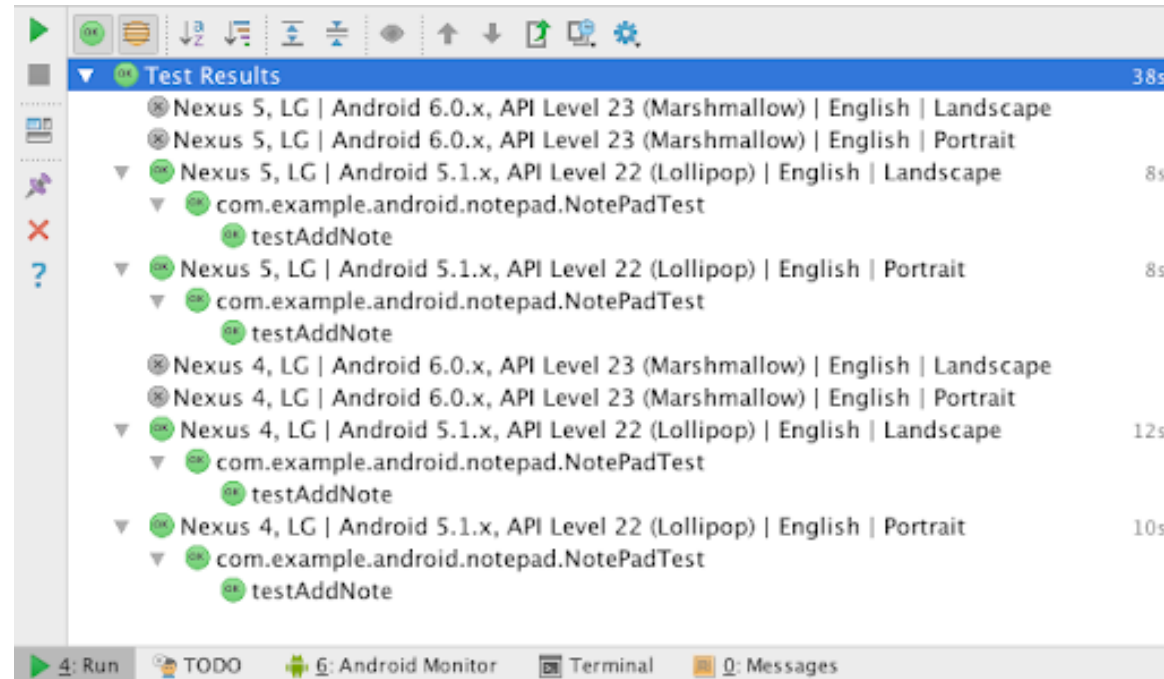
Con Firebase Test Lab, puede probar simultáneamente su aplicación en muchos dispositivos y configuraciones de dispositivos Android populares (diferentes combinaciones de configuración regional, orientación, tamaño de pantalla y versión de plataforma). Estas pruebas se ejecutan en **dispositivos físicos y virtuales en centros de datos remotos de Google**. Los resultados de las pruebas proporcionan registros de pruebas e incluyen los detalles de cualquier falla de la aplicación.

Para comenzar a utilizar Firebase Test Lab, debe hacer lo siguiente:

1. Crea una cuenta de Google , si aún no tienes una.
2. En Firebase console , haz clic en Crear nuevo proyecto .

Analizar los resultados de las pruebas

Cuando Firebase Test Lab complete la ejecución de sus pruebas, se abrirá la **ventana Ejecutar para mostrar los resultados**:

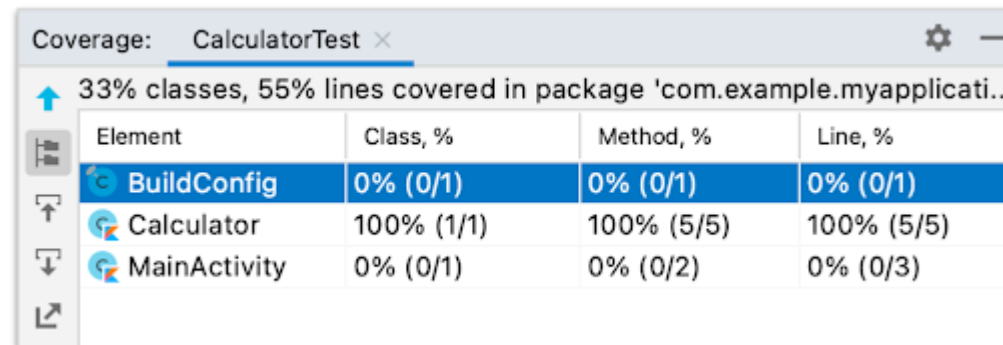


También puede analizar sus pruebas en la web siguiendo el enlace que se muestra al comienzo del registro de ejecución de la prueba en la ventana Ejecutar .

Ver cobertura de prueba

La herramienta de **cobertura de pruebas** está disponible para que las pruebas unitarias locales realicen un seguimiento del **porcentaje y las áreas del código** de su aplicación que han cubierto sus **pruebas unitarias**. Utilice la herramienta de cobertura de prueba para determinar si ha probado adecuadamente los elementos, clases, métodos y líneas de código que componen su aplicación.

Para esto tienes que hacer clic en **Ejecutar nombre-prueba con cobertura** 



Coverage: CalculatorTest x

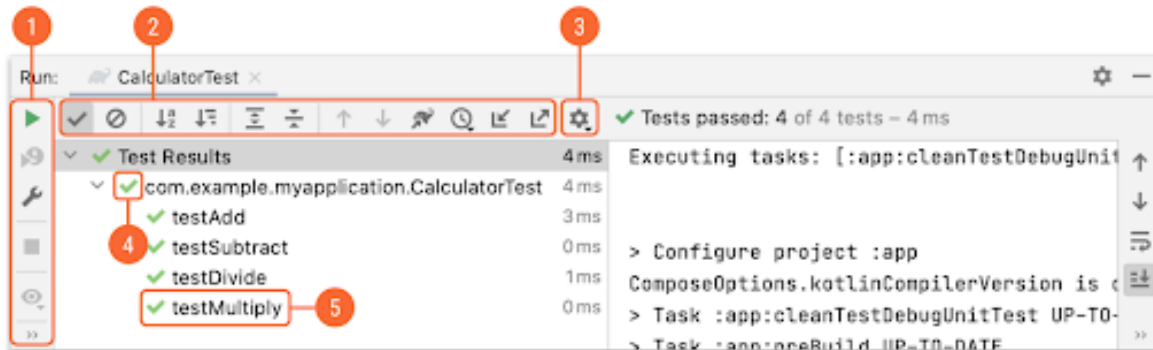
33% classes, 55% lines covered in package 'com.example.myapplication...'

Element	Class, %	Method, %	Line, %
BuildConfig	0% (0/1)	0% (0/1)	0% (0/1)
Calculator	100% (1/1)	100% (5/5)	100% (5/5)
MainActivity	0% (0/1)	0% (0/2)	0% (0/3)

Porcentajes de cobertura de código para una aplicación.

Ver resultados de la prueba

Cuando ejecuta una o más pruebas desde Android Studio, los resultados aparecen en la ventana Ejecutar:



1. Barra de herramientas de ejecución para volver a ejecutar la prueba actual, detener la prueba actual, volver a ejecutar las pruebas fallidas

1. Barra de herramientas de prueba para filtrar y ordenar los resultados de la prueba.

1. Menú contextual para realizar un seguimiento de la prueba en ejecución, mostrar estadísticas en línea, desplazarse hasta el seguimiento de la pila..

1. Los iconos de estado de la prueba indican si una prueba tiene un error, se ignoró, falló, está en progreso, pasó, está en pausa, finalizó o no se ejecutó.

1. Haz clic con el botón derecho en una línea en la vista de árbol para mostrar un menú contextual que le permite ejecutar las pruebas en modo de depuración, abrir el archivo de código fuente de prueba o saltar a la línea del código fuente que se está probando.